

1	Contest	1
	template.cpp	1
	.bashrc	1
	hash.sh	1
2	Mathematics	1
2.1	Equations	2
2.2	Recurrences	2
2.3	Trigonometry	2
2.4	Geometry	2
2.5	Derivatives/Integrals	2
2.6	Sums	2
2.7	Series	2
2.8	Probability theory	3
2.9	Markov chains	3
2.10	Bézout’s identity	3
2.11	Mobius Function	4
	InverseModulo.h	4
	PolyInterpolate.h	4
	BerlekampMassey.h	4
	LinearRecurrence.h	4
	GaussianElimination.h	4
	Lagrange.h	4
	FFTMOD.h	4
	NTTMOD.h	5
	FWHT.h	5
	XorBasis.h	5
	BernoulliNumber.h	6
	LinearSieve.h	6
	Simplex.h	6
	MillerRabin.h	6
	PollardRho.h	6
3	Dynamic Programming	7
	Knuth.h	7
	SlopeTrick.h	7
	AliensTrick.h	7
4	Combinatorial	7
4.1	Permutations	7
4.2	Partitions and subsets	7
4.3	General purpose numbers	7
	PartitionNumbers.h	8
5	Flow - Graph Matching	8
5.1	Modelling with cuts and matchings	8
5.2	Flow with lower bounds $[L, R]$	8
	DinicFlow.h	8
	MaxFlowMinCost.h	9
	GraphFastMatching.h	9
	Hungarian.h	9

	EdmondsBlossom.h	10
	StoerWagner.h	10
	GomoryHuTree.h	10
6	Data structure	10
	ZKWSegTree.h	10
	LiChaoTreePtr.h	10
	DynamicConvexHull.h	11
	SplayTree.h	11
	OrderTree.h	11
	HashMap.h	11
	BitsetTree.h	12
	ManhattanSpanningTree.h	12
	PersistentIT.h	12
	BipartiteDsu.h	12
7	Graph	13
	TwoSat.h	13
	HLD.h	13
	Centroid.h	14
	BlockCutTree.h	14
	OnlineBridgeCounting.h	14
	CheckBridgeArticulation.h	14
	MaximalClique.h	14
	EulerTour.h	15
8	String	15
	KMP.h	15
	Manacher.h	15
	AhoCorasick.h	15
	Hash.h	15
	SuffixArray.h	16
	SuffixArrayDC3.h	16
	MinimalStringRotation.h	16
	LyndonWord.h	16
9	Geometry	17
9.1	Geometric primitives	17
	Point.h	17
	lineDistance.h	17
	SegmentDistance.h	17
	SegmentIntersection.h	17
	lineIntersection.h	17
	sideOf.h	17
	OnSegment.h	17
	linearTransformation.h	17
	Angle.h	17
9.2	Circles	18
	CircleIntersection.h	18
	CircleTangents.h	18
	CirclePolygonIntersection.h	18

	template	.bashrc	hash	1
	circumcircle.h			18
	MinimumEnclosingCircle.h			18
	9.3 Polygons			18
	InsidePolygon.h			18
	PolygonArea.h			18
	PolygonCenter.h			18
	PolygonCut.h			18
	ConvexHull.h			18
	HullDiameter.h			19
	PointInsideHull.h			19
	LineHullIntersection.h			19
	9.4 Misc. Point Set Problems			19
	ClosestPair.h			19
	kdTree.h			19
	FastDelaunay.h			19
	9.5 3D			20
	PolyhedronVolume.h			20
	Point3D.h			20
	3dHull.h			20
	sphericalDistance.h			20
10	Various			20
	yCombinator.h			20
	10.1 Debugging tricks			21
	10.2 Optimization tricks			21
	FastMod.h			21
	FastInput.h			21
	MemoryPool.h			21
	Contest (1)			
	template.cpp			
	<bits/stdc++.h>			920d46, 15 lines
	using namespace std;			
	#define rep(i, a, b) for(int i = a; i < (b); ++i)			
	#define all(x) begin(x), end(x)			
	#define sz(x) (int)(x).size()			
	typedef long long ll;			
	typedef pair<int, int> pii;			
	typedef vector<int> vi;			
	int main(void) {			
	cin.tie(nullptr)->sync_with_stdio(false);			
	cin.exceptions(cin.failbit);			
	return 0;			
	}			
	.bashrc			2 lines
	alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++17 \			
	-fsanitize=undefined,address'			
	hash.sh			3 lines
	# Hashes a file, ignoring all whitespace and comments. Use for			
	# verifying that code was correctly typed.			

```
cpp -dD -P -fpreprocessed | tr -d '[:space:]'| md5sum |cut -c-6
```

Mathematics (2)

2.1 Equations

$$ax^2+bx+c=0\Rightarrow x=\frac{-b\pm\sqrt{b^2-4ac}}{2a}$$

The extremum is given by $x=-b/2a$.

$$\begin{aligned} ax+by=e\quad x&=\frac{ed-bf}{ad-bc} \\ cx+dy=f\Rightarrow y&=\frac{af-ec}{ad-bc} \end{aligned}$$

In general, given an equation $Ax=b$, the solution to a variable x_i is given by

$$x_i=\frac{\det A'_i}{\det A}$$

where A'_i is A with the i 'th column replaced by b .

2.2 Recurrences

If $a_n=c_1a_{n-1}+\cdots+c_ka_{n-k}$, and $r_1,\ldots,r_k$ are distinct roots of $x^k-c_1x^{k-1}-\cdots-c_k$, there are $d_1,\ldots,d_k$ s.t.

$$a_n=d_1r_1^n+\cdots+d_kr_k^n.$$

Non-distinct roots r become polynomial factors, e.g. $a_n=(d_1n+d_2)r^n$.

2.3 Trigonometry

$$\begin{aligned} \sin(v+w)&=\sin v\cos w+\cos v\sin w \\ \cos(v+w)&=\cos v\cos w-\sin v\sin w \end{aligned}$$

$$\begin{aligned} \tan(v+w)&=\frac{\tan v+\tan w}{1-\tan v\tan w} \\ \sin v+\sin w&=2\sin\frac{v+w}{2}\cos\frac{v-w}{2} \\ \cos v+\cos w&=2\cos\frac{v+w}{2}\cos\frac{v-w}{2} \end{aligned}$$

$$(V+W)\tan(v-w)/2=(V-W)\tan(v+w)/2$$

where V,W are lengths of sides opposite angles v,w .

$$\begin{aligned} a\cos x+b\sin x&=r\cos(x-\phi) \\ a\sin x+b\cos x&=r\sin(x+\phi) \end{aligned}$$

where $r=\sqrt{a^2+b^2},\phi=\operatorname{atan2}(b,a)$.

2.4 Geometry

2.4.1 Triangles

Side lengths: a,b,c

Semiperimeter: $p=\frac{a+b+c}{2}$

Area: $A=\sqrt{p(p-a)(p-b)(p-c)}$

Circumradius: $R=\frac{abc}{4A}$

Inradius: $r=\frac{A}{p}$

Length of median (divides triangle into two equal-area triangles):

$$m_a=\tfrac{1}{2}\sqrt{2b^2+2c^2-a^2}$$

Length of bisector (divides angles in two):

$$s_a=\sqrt{bc\left[1-\left(\frac{a}{b+c}\right)^2\right]}$$

Law of sines: $\frac{\sin\alpha}{a}=\frac{\sin\beta}{b}=\frac{\sin\gamma}{c}=\frac{1}{2R}$

Law of cosines: $a^2=b^2+c^2-2bc\cos\alpha$

Law of tangents: $\frac{a+b}{a-b}=\frac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

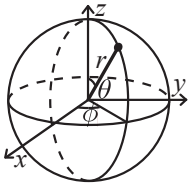
2.4.2 Quadrilaterals

With side lengths a,b,c,d , diagonals e,f , diagonals angle θ , area A and magic flux $F=b^2+d^2-a^2-c^2$:

$$4A=2ef\cdot\sin\theta=F\tan\theta=\sqrt{4e^2f^2-F^2}$$

For cyclic quadrilaterals the sum of opposite angles is 180° , $ef=ac+bd$, and $A=\sqrt{(p-a)(p-b)(p-c)(p-d)}$.

2.4.3 Spherical coordinates



$$\begin{aligned} x&=r\sin\theta\cos\phi & r&=\sqrt{x^2+y^2+z^2} \\ y&=r\sin\theta\sin\phi & \theta&=\operatorname{acos}(z/\sqrt{x^2+y^2+z^2}) \\ z&=r\cos\theta & \phi&=\operatorname{atan2}(y,x) \end{aligned}$$

2.5 Derivatives/Integrals

$$\begin{aligned} \frac{d}{dx}\arcsin x&=\frac{1}{\sqrt{1-x^2}} & \frac{d}{dx}\arccos x&=-\frac{1}{\sqrt{1-x^2}} \\ \frac{d}{dx}\tan x&=1+\tan^2x & \frac{d}{dx}\arctan x&=\frac{1}{1+x^2} \\ \int\tan ax&=-\frac{\ln|\cos ax|}{a} & \int x\sin ax&=\frac{\sin ax-ax\cos ax}{a^2} \\ \int e^{-x^2}&=\frac{\sqrt{\pi}}{2}\operatorname{erf}(x) & \int xe^{ax}dx&=\frac{e^{ax}}{a^2}(ax-1) \end{aligned}$$

Integration by parts:

$$\int_a^bf(x)g(x)dx=[F(x)g(x)]_a^b-\int_a^bF(x)g'(x)dx$$

2.6 Sums

$$c^a+c^{a+1}+\cdots+c^b=\frac{c^{b+1}-c^a}{c-1},c\neq1$$

$$1+2+3+\cdots+n=\frac{n(n+1)}{2}$$

$$1^2+2^2+3^2+\cdots+n^2=\frac{n(2n+1)(n+1)}{6}$$

$$1^3+2^3+3^3+\cdots+n^3=\frac{n^2(n+1)^2}{4}$$

$$1^4+2^4+3^4+\cdots+n^4=\frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

2.7 Series

For a function $f(x)$ that is infinitely differentiable at a point a , the Taylor series centered at a is

$$f(x)=\sum_{n=0}^\infty\frac{f^{(n)}(a)}{n!}(x-a)^n.$$

Setting $a=0$, we obtain the Maclaurin series:

$$f(x)=\sum_{n=0}^\infty\frac{f^{(n)}(0)}{n!}x^n.$$

$$e^x=\sum_{n=0}^\infty\frac{x^n}{n!}.$$

For $|x|<1$,

$$\ln(1+x)=\sum_{n=1}^\infty(-1)^{n-1}\frac{x^n}{n}.$$

$$\sin x = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n+1}}{(2n+1)!}.$$

$$\cos x = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n}}{(2n)!}.$$

$$(1+x)^k = \sum_{n=0}^{\infty} \binom{k}{n} x^n, \quad |x| < 1$$

$$\binom{k}{n} = \frac{k(k-1)(k-2)\cdots(k-n+1)}{n!} = \frac{(k)_n}{n!}.$$

2.8 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x . It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x x p_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y ,

$$V(aX + bY) = a^2V(X) + b^2V(Y).$$

2.8.1 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\text{Bin}(n, p)$, $n = 1, 2, \dots$, $0 \leq p \leq 1$.

$$p(k) = \binom{n}{k} p^k (1-p)^{n-k}$$

$$\mu = np, \sigma^2 = np(1-p)$$

$\text{Bin}(n, p)$ is approximately $\text{Po}(np)$ for small p .

Hockey-stick identity

$$\sum_{i=r}^n \binom{i}{r} = \binom{n+1}{r+1} \quad \text{for } n, r \in \mathbb{N}, n \geq r$$

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $\text{Fs}(p)$, $0 \leq p \leq 1$.

$$p(k) = p(1-p)^{k-1}, \quad k = 1, 2, \dots$$

$$\mu = \frac{1}{p}, \sigma^2 = \frac{1-p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $\text{Po}(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda} \frac{\lambda^k}{k!}, \quad k = 0, 1, 2, \dots$$

$$\mu = \lambda, \sigma^2 = \lambda$$

2.8.2 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $\text{U}(a, b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a+b}{2}, \sigma^2 = \frac{(b-a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $\text{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda}, \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ are independent, then

$$aX_1 + bX_2 + c \sim \mathcal{N}(a\mu_1 + b\mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

2.9 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. Let $X_1, X_2, \dots$ be a sequence of random variables generated by the Markov process. Then there is a transition matrix $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, and $\mathbf{p}^{(n)} = \mathbf{P}^n \mathbf{p}^{(0)}$ is the probability distribution for X_n (i.e., $p_i^{(n)} = \Pr(X_n = i)$), where $\mathbf{p}^{(0)}$ is the initial distribution.

π is a stationary distribution if $\pi = \mathbf{P}\pi$. If the Markov chain is *irreducible* (it is possible to get to any state from any state), then $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state i . π_j/π_i is the expected number of visits in state j between two visits in state i .

For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, π_i is proportional to node i 's degree.

A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k \rightarrow \infty} \mathbf{P}^k = \pi \mathbf{1}^T$.

A Markov chain is an **A-chain** if the states can be partitioned into two sets **A** and **G**, such that all states in **A** are absorbing ($p_{ii} = 1$), and all states in **G** leads to an absorbing state in **A**. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is j , is $a_{ij} = p_{ij} + \sum_{k \in \mathbf{G}} a_{ik} p_{kj}$. The expected time until absorption, when the initial state is i , is $t_i = 1 + \sum_{k \in \mathbf{G}} p_{ki} t_k$.

2.10 Bézout's identity

For $a \neq 0$, $b \neq 0$, then $d = \text{gcd}(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\text{gcd}(a, b)}, y - \frac{ka}{\text{gcd}(a, b)} \right), \quad k \in \mathbb{Z}$$

2.11 Mobius Function

μ(n) = { 0 n is not square free; 1 n has even number of prime factors; -1 n has odd number of prime factors

Mobius Inversion:

g(n) = ∑_{d|n} f(d) ⇔ f(n) = ∑_{d|n} μ(d)g(n/d)

Other useful formulas/forms:

∑_{d|n} μ(d) = [n = 1] (very useful)

g(n) = ∑_{n|d} f(d) ⇔ f(n) = ∑_{n|d} μ(d/n)g(d)

g(n) = ∑_{1≤m≤n} f(⌊n/m⌋) ⇔ f(n) = ∑_{1≤m≤n} μ(m)g(⌊n/m⌋)

InverseModulo.h

Description: Inverse of a modulo m (m need not be prime) via extended Euclid. Requires gcd(a, m) = 1. Returns x in [0, m). Time: O(log m)

```
template<class T> T inverse_modulo(T a, T m) {
    T m0 = m, u = 0, v = 1;
    a %= m; if (a < 0) a += m;
    while (a > 0) {
        T t = m / a;
        m -= t * a; swap(a, m);
        u -= t * v; swap(u, v);
    }
    assert(m == 1);
    return u < 0 ? u + m0 : u;
}
```

PolyInterpolate.h

Description: Given n points (x[i], y[i]), computes an n-1-degree polynomial p that passes through them: p(x) = a[0] * x^0 + ... + a[n-1] * x^{n-1}. For numerical precision, pick x[k] = c * cos(k/(n-1) * π), k = 0...n-1. Time: O(n^2)

```
typedef vector<double> vd;
vd interpolate(vd x, vd y, int n) {
    vd res(n), temp(n);
    rep(k,0,n-1) rep(i,k+1,n)
        y[i] = (y[i] - y[k]) / (x[i] - x[k]);
    double last = 0; temp[0] = 1;
    rep(k,0,n) rep(i,0,n) {
        res[i] += y[k] * temp[i];
        swap(last, temp[i]);
        temp[i] -= last * x[k];
    }
    return res;
}
```

BerlekampMassey.h

Description: Recovers any n-order linear recurrence relation from the first 2n terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size ≤ n. Needs prime const ll mod and modpow(b, e) (ModPow.h, not printed). Usage: berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2} Time: O(N^2)

```
"ModPow.h"
vector<ll> berlekampMassey(vector<ll> s) {
    int n = sz(s), L = 0, m = 0;
```

```
vector<ll> C(n + 1), B(n + 1), T;
C[0] = B[0] = 1;

ll b = 1;
rep(i,0,n) { ++m;
    ll d = s[i] % mod;
    rep(j,1,L+1) d = (d + C[j] * s[i - j]) % mod;
    if (!d) continue;
    T = C; ll coef = d * modpow(b, mod-2) % mod;
    rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;
    if (2 * L > i) continue;
    L = i + 1 - L; B = T; b = d; m = 0;
}

C.resize(L + 1); C.erase(C.begin());
for (ll& x : C) x = (mod - x) % mod;
return C;
}
```

LinearRecurrence.h

Description: Generates the k'th term of an n-order linear recurrence S[i] = ∑_j S[i - j - 1]tr[j], given S[0... ≥ n - 1] and tr[0...n - 1]. Faster than matrix multiplication. Useful together with Berlekamp-Massey. Too slow for n > 10^4. Needs const ll mod and modpow(b, e) (ModPow.h, not printed). Usage: linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci number Time: O(n^2 log k)

```
"ModPow.h"
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
    int n = sz(tr);

    auto combine = [&](Poly a, Poly b) {
        Poly res(n * 2 + 1);
        rep(i,0,n+1) rep(j,0,n+1)
            res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
        for (int i = 2 * n; i > n; --i) rep(j,0,n)
            res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
        res.resize(n + 1);
        return res;
    };

    Poly pol(n + 1), e(pol);
    pol[0] = e[1] = 1;

    for (++k; k; k /= 2) {
        if (k % 2) pol = combine(pol, e);
        e = combine(e, e);
    }

    ll res = 0;
    rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
    return res;
}
```

GaussianElimination.h

Description: Solves Ax = b over doubles. Input is the augmented n × (m+1) matrix [A|b]. Returns false if inconsistent; otherwise x is one solution (free variables set to 0). Usage: vector<double> x; gauss({{1, 2, 3}, {4, 5, 6}}, x); // x = {-1, 2} Time: O(NM min(N, M))

```
const double EPS = 1e-9;
bool gauss(vector<vector<double>> a, vector<double>& x) {
    int n = sz(a), m = n ? sz(a[0]) - 1 : 0;
    vi pivot(m, -1);
    for (int col = 0, row = 0; col < m && row < n; col++) {
```

```
int cur = row;
rep(i,row,n) if (fabs(a[i][col]) > fabs(a[cur][col])) cur = i;
if (fabs(a[cur][col]) < EPS) continue;
swap(a[cur], a[row]);
pivot[col] = row;
rep(i,0,n) if (i != row && fabs(a[i][col]) > EPS) {
    double c = a[i][col] / a[row][col];
    rep(j,col,m+1) a[i][j] -= a[row][j] * c;
}
row++;
}
x.assign(m, 0);
rep(i,0,m) if (pivot[i] != -1)
    x[i] = a[pivot[i]][m] / a[pivot[i]][i];
rep(i,0,n) {
    double s = a[i][m];
    rep(j,0,m) s -= x[j] * a[i][j];
    if (fabs(s) > EPS) return false;
}
return true;
}
```

Lagrange.h

Description: Given p[i] = f(i) for i ∈ [0, n] with deg f < n, returns f(x) for any integer x (negative or huge OK). Z is a mod-P int type (Z(ll) reducing negatives, + - * / +=), prime P > n. Usage: lagrange<Z>(p, x) Time: O(n log P)

```
template<class Z> Z lagrange(const vector<Z>& p, ll x) {
    int n = sz(p);
    vector<Z> pre(n + 1, 1), suf(n + 1, 1), fi(n, 1);
    rep(i,0,n) pre[i+1] = pre[i] * Z(x - i);
    for (int i = n; i--;) suf[i] = suf[i+1] * Z(x - i);
    rep(i,1,n) fi[i] = fi[i-1] * Z(i);
    Z ans = 0;
    rep(i,0,n) {
        Z t = p[i] * pre[i] * suf[i+1] / (fi[i] * fi[n-1-i]);
        ans += (n - 1 - i) % 2 ? Z(0) - t : t;
    }
    return ans;
}
```

FFTMOD.h

Description: Convolution of arbitrary ll vectors modulo any MOD, via 2 FFTs of size ≥ |a| + |b| splitting at √MOD. Inputs must be in [0, MOD). Precise for MOD ≤ 10^9 and N up to ~ 10^6; wrong for MOD near 2^50. Usage: convMod<1000000007>(a, b) Time: O(N log N)

```
typedef complex<double> C;
typedef long double ld;
void fft(vector<C> &a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vector<complex<ld>> R(2, 1);
    static vector<C> rt(2, 1); // (^ 10% faster if double)
    for (int k = 2; k < n; k *= 2) {
        R.resize(n), rt.resize(n);
        auto x = polar(1.0L, acos(-1.0L) / k);
        for (int i = k; i < 2 * k; ++i) rt[i] = R[i] = (i & 1) ? R[i / 2] * x : R[i / 2];
    }
    vector<int> rev(n);
    for (int i = 0; i < n; ++i) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    for (int i = 0; i < n; ++i) if(i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k <= 1) {
```

```
for (int i = 0; i < n; i += k << 1) {
    for (int j = 0; j < k; ++j) {
        auto x = (double*) &rt[j + k], y = (double*) &a[i + j + k];
        C z(x[0] * y[0] - x[1] * y[1], x[0] * y[1] + x[1] * y[0]);
        a[i + j + k] = a[i + j] - z; a[i + j] += z;
    }
}

template<ll MOD> vector<ll> convMod(const vector<ll> &a, const vector<ll> &b) {
    if (a.empty() || b.empty()) return {};
    vector<ll> res(sz(a) + sz(b) - 1, 0);
    int B = 32 - __builtin_clz(sz(res));
    int n = 1 << B, cut = ll(sqrt(MOD));
    vector<C> L(n), R(n), outs(n), outl(n);
    for (int i = 0; i < sz(a); ++i) L[i] = C(ll(a[i] / cut), ll(a[i] % cut));
    for (int i = 0; i < sz(b); ++i) R[i] = C(ll(b[i] / cut), ll(b[i] % cut));
    fft(L), fft(R);
    for (int i = 0; i < n; ++i) {
        int j = -i & (n - 1);
        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / C(1.0i);
    }
    fft(outl), fft(outs);
    for (int i = 0; i < sz(res); ++i) {
        ll av = ll(real(outl[i]) + .5), cv = ll(imag(outs[i]) + .5);
        ll bv = ll(imag(outl[i]) + .5) + ll(real(outs[i]) + .5);
        res[i] = ((av % MOD * cut % MOD + bv) % MOD * cut % MOD + cv) % MOD;
    }
    return res;
}
```

NTTMOD.h
Description: Convolution modulo MOD = 998244353 (= 119·2²³+1, primitive root 3). Inputs may be any ll; they are reduced mod MOD. For other NTT primes $c \cdot 2^k + 1$ change MOD and G.
Usage: convolution({1, 2}, {3, 4}) // {3, 10, 8}
Time: $\mathcal{O}(N \log N)$

642579, 48 lines

```
const ll MOD = 998244353, G = 3;
ll mod_pow(ll a, ll e) {
    ll r = 1;
    for (a %= MOD; e; e >>= 1, a = a * a % MOD)
        if (e & 1) r = r * a % MOD;
    return r;
}
```

```
void ntt(vector<ll> &a, bool invert) {
    int n = sz(a);
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <= 1) {
        ll wlen = mod_pow(G, (MOD - 1) / len);
        if (invert) wlen = mod_pow(wlen, MOD - 2);
        for (int i = 0; i < n; i += len) {
            ll w = 1;
            rep(j, 0, len/2) {
```

```
                ll u = a[i+j], v = a[i+j+len/2] * w % MOD;
                a[i+j] = u + v < MOD ? u + v : u + v - MOD;
                a[i+j+len/2] = u - v >= 0 ? u - v : u - v + MOD;
                w = w * wlen % MOD;
            }
        }
    }
    if (invert) {
        ll n_inv = mod_pow(n, MOD - 2);
        for (ll &x : a) x = x * n_inv % MOD;
    }
}
```

```
vector<ll> convolution(const vector<ll> &a, const vector<ll> &b) {
    if (!sz(a) || !sz(b)) return {};
    vector<ll> fa(all(a)), fb(all(b));
    for (ll &x : fa) x = (x % MOD + MOD) % MOD;
    for (ll &x : fb) x = (x % MOD + MOD) % MOD;
    int n = 1, need = sz(a) + sz(b) - 1;
    while (n < need) n <= 1;
    fa.resize(n), fb.resize(n);
    ntt(fa, false), ntt(fb, false);
    rep(i, 0, n) fa[i] = fa[i] * fb[i] % MOD;
    ntt(fa, true), fa.resize(need);
    return fa;
}
```

FWHT.h
Description: Xor-convolution modulo prime MOD: $c_k = \sum_{i \oplus j = k} a_i b_j$. sz(a) must be a power of 2. For AND use (u+v, v) / (u-v, v); for OR use (u, u+v) / (u, v-u), without the final 1/n scaling.
Usage: FWHT(a), FWHT(b); a[i] = a[i]*b[i] mod MOD; FWHT(a, true);
Time: $\mathcal{O}(N \log N)$

1e9c8c, 19 lines

```
const ll MOD = 998244353;
ll pw(ll b, ll e) {
    ll r = 1;
    for (b %= MOD; e; e >>= 1, b = b * b % MOD)
        if (e & 1) r = r * b % MOD;
    return r;
}
void FWHT(vector<ll> &a, bool inv = false) {
    int n = sz(a);
    for (int k = 1; k < n; k <= 1)
        for (int i = 0; i < n; i += 2 * k) rep(j, i, i+k) {
            ll u = a[j], v = a[j + k];
            a[j] = (u + v) % MOD, a[j + k] = (u - v + MOD) % MOD;
        }
    if (inv) {
        ll ni = pw(n, MOD - 2);
        for (ll &x : a) x = x * ni % MOD;
    }
}
```

XorBasis.h
Description: Linear basis of non-negative ints over GF(2). query(k): k-th smallest (1-indexed) value of the span, $k \leq 2^{szBasis}$. getValPos(x): 1-indexed rank of x in the span; x MUST be in the span. mergeNode: basis of the sum of two spans (e.g. segment tree nodes).
Time: $\mathcal{O}(\log C)$ per insert/check, $\mathcal{O}(\log^2 C)$ per merge

b2d259, 76 lines

```
const int MAXLOG = 31;

struct Node {
    int basis[MAXLOG], szBasis, basisMask;

    Node() {
```

```
        szBasis = basisMask = 0;
        memset(basis, 0, sizeof(basis));
    }

    bool insertVector(int mask) {
        while(mask > 0) {
            int i = 31 - __builtin_clz(mask);
            if(!basis[i]) {
                ++szBasis, basis[i] = mask;
                basisMask |= (1 << i);
                return true;
            }
            mask ^= basis[i];
        }
        return false;
    }
}
```

```
bool checkVector(int mask) const {
    while(mask > 0) {
        int i = 31 - __builtin_clz(mask);
        if(!basis[i]) return false;
        mask ^= basis[i];
    }
    return true;
}
```

```
Node mergeNode(const Node &A) {
    Node res(*this);
    int mask = A.basisMask;
    while(mask > 0) {
        int i = 31 - __builtin_clz(mask);
        res.insertVector(A.basis[i]);
        mask ^= (1 << i);
    }

    return res;
}
```

```
// find k-th element from small to big in Basis
int query(int k) const { // 1-indexed
    int mask = 0, tot = 1 << szBasis, fMask = basisMask;
    while(fMask > 0) {
        int i = 31 - __builtin_clz(fMask), low(tot >> 1);
        if((low < k) != (mask >> i & 1)) mask ^= basis[i];
        if(low < k) k -= low;
        fMask ^= (1 << i), tot >>= 1;
    }
    return mask;
}
```

```
int getValPos(int mask) const {
    int firstMask = mask, fMask = basisMask, tot(1 << szBasis), cnt(1);
    while(fMask > 0) {
        int i = 31 - __builtin_clz(fMask), low(tot >> 1);
        if(firstMask >> i & 1) cnt += low;
        fMask ^= (1 << i), tot >>= 1;
    }
    return cnt;
}
```

```
int getMax(void) const {
    int res = 0, mask = basisMask;
    while(mask > 0) {
        int i = 31 - __builtin_clz(mask);
        if(!(res >> i & 1)) res ^= basis[i];
        mask ^= (1 << i);
    }
    return res;
}
```

```
    }
};
```

BernoulliNumber.h

Description: Bernoulli numbers $B_0..B_K$ with $B_1 = +\frac{1}{2}$ (the B^+ convention; negate $B[1]$ for B^-), and $\text{powerSum} = \sum_{i=1}^n i^k = \frac{1}{k+1} \sum_{j=0}^k \binom{k+1}{j} B_j n^{k+1-j}$. Z is a modular type mod- P int ($Z(l1)$, $+ - * / +=$), prime $P > K + 1$.

Usage: auto B = bernoulli<Z>(K); Z s = powerSum(n, k, B); // k<=K

Time: $\mathcal{O}(K^2)$ for bernoulli, $\mathcal{O}(k \log P)$ for powerSum

```
template<class Z> vector<Z> bernoulli(int K) {
    vector<Z> B(K + 1), f(K + 2, 1), fi(K + 2);
    rep(i,1,K+2) f[i] = f[i-1] * Z(i);
    fi[K+1] = Z(1) / f[K+1];
    for (int i = K + 1; i; i--) fi[i-1] = fi[i] * Z(i);
    B[0] = 1;
    rep(m,1,K+1) {
        Z s = 0;
        rep(j,0,m) s += f[m+1] * fi[j] * fi[m+1-j] * B[j];
        B[m] = Z(0) - s / Z(m + 1);
    }
    if (K) B[1] = Z(0) - B[1];
    return B;
}
```

```
template<class Z> Z powerSum(l1 n, int k, const vector<Z>& B) {
    vector<Z> pw(k + 2, 1);
    rep(i,1,k+2) pw[i] = pw[i-1] * Z(n);
    Z res = 0, c = 1; // c = C(k+1, j)
    rep(j,0,k+1) {
        res += c * B[j] * pw[k+1-j];
        c = c * Z(k + 1 - j) / Z(j + 1);
    }
    return res / Z(k + 1);
}
```

LinearSieve.h

Description: Linear sieve for all $x < \text{MAXN}$: Mobius u, number of divisors numd, Euler phi, and cnt = exponent of the smallest prime.

Time: $\mathcal{O}(\text{MAXN})$

```
const int MAXN = 1e6 + 5;
int u[MAXN], numd[MAXN], phi[MAXN], cnt[MAXN];
bool isComp[MAXN];
vi primes;
void linearSieve() {
    u[1] = numd[1] = phi[1] = 1;
    rep(i,2,MAXN) {
        if (!isComp[i]) {
            primes.push_back(i);
            u[i] = -1, numd[i] = 2, phi[i] = i - 1, cnt[i] = 1;
        }
        for (int p : primes) {
            if ((l1)i * p >= MAXN) break;
            int x = i * p;
            isComp[x] = 1;
            if (i % p == 0) {
                u[x] = 0, cnt[x] = cnt[i] + 1;
                numd[x] = numd[i] / (cnt[i] + 1) * (cnt[i] + 2);
                phi[x] = phi[i] * p;
                break;
            }
            u[x] = -u[i], numd[x] = numd[i] * 2;
            phi[x] = phi[i] * (p - 1), cnt[x] = 1;
        }
    }
}
```

Simplex.h

Description: Two-phase simplex. Solves $\max c^T x$ s.t. $Ax \leq b, x \geq 0$ (A is $m \times n$). Solve(x) returns the optimum and stores an optimal x; returns inf if unbounded, -inf if infeasible.

Usage: LPSolver lp(A, b, c); VD x; DOUBLE v = lp.Solve(x);

Time: $\mathcal{O}(NM)$ per pivot, exponential worst case

```
typedef long double DOUBLE;
typedef vector<DOUBLE> VD;
typedef vector<VD> VVD;
typedef vector<int> VI;
```

const DOUBLE EPS = 1e-9;

```
struct LPSolver {
    int m, n;
    VI B, N;
    VVD D;

    LPSolver(const VVD &A, const VD &b, const VD &c) :
        m(b.size()), n(c.size()), B(m), N(n + 1), D(m + 2, VD(n + 2)) {
        for (int i = 0; i < m; i++) for (int j = 0; j < n; j++) D
            [i][j] = A[i][j];
        for (int i = 0; i < m; i++) { B[i] = n + i; D[i][n] = -1;
            D[i][n + 1] = b[i]; }
        for (int j = 0; j < n; j++) { N[j] = j; D[m][j] = -c[j];
            }
        N[n] = -1; D[m + 1][n] = 1;
    }
}
```

```
void Pivot(int r, int s) {
    for (int i = 0; i < m + 2; i++) if (i != r)
        for (int j = 0; j < n + 2; j++) if (j != s)
            D[i][j] -= D[r][j] * D[i][s] / D[r][s];
    for (int j = 0; j < n + 2; j++) if (j != s) D[r][j] /= D[r
        ][s];
    for (int i = 0; i < m + 2; i++) if (i != r) D[i][s] /= -D[r
        ][s];
    D[r][s] = 1.0 / D[r][s];
    swap(B[r], N[s]);
}
```

```
bool Simplex(int phase) {
    int x = phase == 1 ? m + 1 : m;
    while (true) {
        int s = -1;
        for (int j = 0; j <= n; j++) {
            if (phase == 2 && N[j] == -1) continue;
            if (s == -1 || D[x][j] < D[x][s] || (D[x][j] == D[x][s]
                && N[j] < N[s])) s = j;
        }
        if (D[x][s] > -EPS) return true;
        int r = -1;
        for (int i = 0; i < m; i++) {
            if (D[i][s] < EPS) continue;
            if (r == -1 || D[i][n + 1] / D[i][s] < D[r][n + 1] / D[
                r][s] || ((D[i][n + 1] / D[i][s]) == (D[r][n + 1]
                    / D[r][s]) && B[i] < B[r])) r = i;
        }
        if (r == -1) return false;
        Pivot(r, s);
    }
}
```

```
DOUBLE Solve(VD &x) {
    int r = 0;
    for (int i = 1; i < m; i++) if (D[i][n + 1] < D[r][n + 1])
        r = i;
```

```
    if (D[r][n + 1] < -EPS) {
        Pivot(r, n);
        if (!Simplex(1) || D[m + 1][n + 1] < -EPS) return -
            numeric_limits<DOUBLE>::infinity();
        for (int i = 0; i < m; i++) if (B[i] == -1) {
            int s = -1;
            for (int j = 0; j <= n; j++)
                if (s == -1 || D[i][j] < D[i][s] || (D[i][j] == D[i][
                    s] && N[j] < N[s])) s = j;
            Pivot(i, s);
        }
    }
    if (!Simplex(2)) return numeric_limits<DOUBLE>::infinity();
    x = VD(n);
    for (int i = 0; i < m; i++) if (B[i] < n) x[B[i]] = D[i][n
        + 1];
    return D[m][n + 1];
}
};
```

MillerRabin.h

Description: Deterministic Miller-Rabin for all $n < 2^{64}$. Bases {2,7,61} suffice for $n < 2^{32}$, the 7 bases below for $n < 2^{64}$. Other known sets: $n < 2047$: {2}; $n < 9\,080\,191$: {31,73}; $n < 3\,474\,749\,660\,383$: {2,3,5,7,11,13}; $n < 3.3 \cdot 10^{24}$: first 13 primes.

Time: $\mathcal{O}(A \log N)$, A = number of bases

```
typedef unsigned long long ull;
ull mulmod(ull a, ull b, ull M) { return (__uint128_t)a * b % M
    ; }
ull powermod(ull a, ull e, ull M) {
    ull r = 1;
    for (a %= M; e; e >>= 1, a = mulmod(a, a, M))
        if (e & 1) r = mulmod(r, a, M);
    return r;
}
```

```
bool check_miller(int s, ull d, ull n, ull a) {
    if (a % n == 0) return true;
    ull p = powermod(a, d, n);
    if (p == 1) return true;
    for (; s > 0; --s) {
        if (p == n - 1) return true;
        p = mulmod(p, p, n);
        if (p == 1) return false;
    }
    return false;
}
```

```
bool isPrime(ull n) {
    for (ull p : {2, 3, 5, 7}) if (n % p == 0) return n == p;
    if (n < 121) return n > 1;
    int s = __builtin_ctzll(n - 1);
    ull d = (n - 1) >> s;
    if (n < (1ULL << 32)) {
        for (ull a : {2, 7, 61})
            if (!check_miller(s, d, n, a)) return false;
    } else {
        for (ull a : {2, 325, 9375, 28178, 450775, 9780504,
            1795265022})
            if (!check_miller(s, d, n, a)) return false;
    }
    return true;
}
```

PollardRho.h

Description: Prime factorization of $1 \leq n < 2^{63}$ (sorted, with multiplicity) using Pollard rho with Brent's cycle detection and batched gcds. factorize(1) = {}.

```
Time:  $\mathcal{O}\left(n^{1/4}\log^2 n\right)$ 
"MillerRabin.h" 3366d3, 47 lines
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

void pollard(ll n, vector<ll> &ans) {
    if (n == 1) return;
    if (isPrime(n)) { ans.push_back(n); return; }
    while (1) {
        ll c = 1 + rng() % (n - 1);
        auto f = [&](ll y) {
            ull r = mulmod(y, y, n) + c;
            return ll(r >= (ull)n ? r - n : r);
        };
        ll y = 2, g = 1;
        int B = 100, len = 1;
        while (1) {
            ll z = y, zs = -1;
            rep(i, 0, len) z = f(z);
            int lft = len;
            while (g == 1 && lft > 1) {
                zs = z;
                ll p = 1;
                for (int i = 0; i < B && i < lft; ++i) {
                    p = mulmod(p, abs(z - y), n);
                    z = f(z);
                }
                g = __gcd(p, n);
                lft -= B;
            }
            if (g == 1) { y = z; len <= 1; continue; }
            if (g == n) {
                g = 1, z = zs;
                while (g == 1) g = __gcd(abs(z - y), n), z = f(z);
            }
            if (g == n) break;
            pollard(g, ans), pollard(n / g, ans);
            return;
        }
    }
}

vector<ll> factorize(ll n) {
    vector<ll> ans;
    for (ll p : {2, 3, 5, 7, 11, 13, 17, 19})
        while (n % p == 0) n /= p, ans.push_back(p);
    pollard(n, ans);
    sort(all(ans));
    return ans;
}
```

Dynamic Programming (3)

```
Knuth.h
Description: Knuth optimization for  $dp[i][j] = \min_{i \leq k < j} dp[i][k] + dp[k+1][j] + C(i, j)$ . Valid when C satisfies the quadrangle inequality  $C(a, c) + C(b, d) \leq C(a, d) + C(b, c)$  ( $a \leq b \leq c \leq d$ ) and is monotone on inclusion; then  $opt[i][j-1] \leq opt[i][j] \leq opt[i+1][j]$ .  $base[i] = dp[i][i]$ . 0-indexed.
Time:  $\mathcal{O}(N^2)$ 
1ba7d9, 13 lines
template<class F> ll knuth(int n, vector<ll> base, F C) {
    vector<vector<ll>> dp(n, vector<ll>(n));
    vector<vi> opt(n, vi(n));
    rep(i, 0, n) dp[i][i] = base[i], opt[i][i] = i;
    for (int i = n - 2; i >= 0; --i) rep(j, i+1, n) {
        ll mn = LLONG_MAX, c = C(i, j);
        for (int k = opt[i][j-1]; k <= min(j-1, opt[i+1][j]); ++k)
```

```
        if (dp[i][k] + dp[k+1][j] + c <= mn)
            mn = dp[i][k] + dp[k+1][j] + c, opt[i][j] = k;
        dp[i][j] = mn;
    }
    return dp[0][n-1];
}
```

```
SlopeTrick.h
Description: Slope trick example: minimum total —change— (+1/-1 operations) to make A strictly increasing. For non-decreasing, drop the "- i". The multiset holds the breakpoints of the convex function; its max is the argmin.
Time:  $\mathcal{O}(N \log N)$ 
9e40ff, 13 lines
```

```
ll slopeTrick(const vector<ll>& A) {
    multiset<ll> pts; ll ans = 0;
    rep(i, 0, sz(A)) {
        ll a = A[i] - i;
        pts.insert(a);
        ll opt = *pts.rbegin();
        if (a < opt) {
            pts.erase(prev(pts.end()));
            pts.insert(a), ans += opt - a;
        }
    }
    return ans;
}
```

```
AliensTrick.h
Description: Aliens / Lagrangian relaxation. Maximize f(k) where f(k) = best value using exactly k items and f is concave. calc(lam) must solve the problem where each item costs an extra lam, returning best value, max count among optimal solutions. For minimization negate values.
Usage: aliens(k, [&](ll lam) { ...; return pair{v, c}; })
Time:  $\mathcal{O}(\log C)$  calls to calc.
833a39, 10 lines
```

```
template<class F> ll aliens(ll k, F calc) {
    ll lo = -1e12, hi = 1e12, ans = 0; // |lam| > max slope
    while (lo <= hi) {
        ll mid = lo + (hi - lo) / 2;
        auto [v, c] = calc(mid);
        if (c >= k) ans = v + mid * k, lo = mid + 1;
        else hi = mid - 1;
    }
    return ans;
}
```

Combinatorial (4)

4.1 Permutations

4.1.1 Cycles

Let $g_S(n)$ be the number of n -permutations whose cycle lengths all belong to the set S . Then

$$\sum_{n=0}^{\infty} g_S(n) \frac{x^n}{n!} = \exp \left(\sum_{n \in S} \frac{x^n}{n} \right)$$

4.1.2 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rfloor$$

4.1.3 Burnside’s lemma

Given a group G of symmetries and a set X , the number of elements of X up to symmetry equals

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts “configurations” (of some sort) of length n , we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n, k)) = \frac{1}{n} \sum_{k|n} f(k) \phi(n/k).$$

4.2 Partitions and subsets

4.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1, \quad p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k - 1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	$\sim 2\text{e}5$	$\sim 2\text{e}8$

4.2.2 Lucas’ Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + \dots + n_1 p + n_0$ and $m = m_k p^k + \dots + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^k \binom{n_i}{m_i} \pmod{p}$.

4.3 General purpose numbers

4.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t - 1}$ (FFT-able), which gives $B_1 = -\frac{1}{2}$ (used below; BernoulliNumber.h uses $+\frac{1}{2}$). $B[0, \dots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \dots]$

Sums of powers:

$$\sum_{i=1}^n i^m = \frac{1}{m+1} \sum_{k=0}^m \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\begin{aligned} \sum_{i=m}^{\infty} f(i) &= \int_m^{\infty} f(x) dx - \sum_{k=1}^{\infty} \frac{B_k}{k!} f^{(k-1)}(m) \\ &\approx \int_m^{\infty} f(x) dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m)) \end{aligned}$$

4.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k), \quad c(0,0) = 1$$
$$\sum_{k=0}^n c(n,k)x^k = x(x+1)\dots(x+n-1)$$

$$c(8,k) = 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$$
$$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \dots$$

4.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j :s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j :s s.t. $\pi(j) \geq j$, k j :s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$
$$E(n,0) = E(n,n-1) = 1$$
$$E(n,k) = \sum_{j=0}^k (-1)^j \binom{n+1}{j} (k+1-j)^n$$

4.3.4 Motzkin numbers

- different ways of drawing non-intersecting chords between n points on a circle.
- positive** integer sequences of length $n+1$ in which the opening and ending elements are 1, and the difference between any two consecutive elements is -1, 0 or 1.
- 1, 1, 2, 4, 9, 21, 51, 127, 323, 835, ...

$$M_n = M_{n-1} + \sum_{i=0}^{n-2} M_i M_{n-2-i} = \frac{2n+1}{n+2} M_{n-1} + \frac{3n-3}{n+2} M_{n-2}$$

4.3.5 Delannoy numbers

- Number of ways to move from (0,0) to (m,n) with 3 kinds of steps: (1,0), (1,1), (0,1)
- $D(0,k) = 1, 1, 1, 1, \dots$
- $D(3,k) = 1, 7, 25, 63, \dots$

$$D(m,n) = \sum_{k=0}^{\min(m,n)} \binom{m+n-k}{m} \binom{m}{k} = \sum_{k=0}^{\min(m,n)} \binom{m}{k} \binom{n}{k} 2^k$$

4.3.6 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$
$$S(n,1) = S(n,n) = 1$$
$$S(n,k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

4.3.7 Bell numbers

Number of partitions of a set of n items into non-empty disjoint subsets. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$. For p prime,

$$B_{n+1} = \sum_{k=0}^n \binom{n}{k} B_k$$
$$B_{p+n} \equiv B_n + B_{n+1} \pmod{p}$$
$$B_{p^m+n} \equiv mB_n + B_{n+1} \pmod{p}$$

4.3.8 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i : $n_1 n_2 \dots n_k n^{k-2}$
with degrees d_i : $(n-2)! / ((d_1-1)! \dots (d_n-1)!)$

4.3.9 Catalan numbers

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$
$$C_0 = 1, \quad C_{n+1} = \frac{2(2n+1)}{n+2} C_n, \quad C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- full binary trees with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.
- correct bracket sequence of length $2n$.

PartitionNumbers.h

Description: $p(0..n) \bmod \text{MOD}$, $p(i)$ = number of ways to write i as a sum of positive integers, by Euler's pentagonal theorem.
Time: $\mathcal{O}(n\sqrt{n})$

```
vector<ll> partitions(int n, ll MOD = 998244353) {
    vector<ll> dp(n + 1);
    dp[0] = 1;
    rep(i,1,n+1) for (int k = 1; ; ++k) {
        int g1 = k * (3 * k - 1) / 2, g2 = k * (3 * k + 1) / 2;
        if (g1 > i) break;
        ll s = dp[i - g1] + (g2 <= i ? dp[i - g2] : 0);
        dp[i] = (dp[i] + (k % 2 ? s : MOD - s % MOD)) % MOD;
    }
    return dp;
}
```

Flow - Graph Matching (5)

5.1 Modelling with cuts and matchings

- Hall:** bipartite $(L \cup R, E)$ has a matching saturating L iff $|N(S)| \geq |S|$ for all $S \subseteq L$.

- König:** in bipartite graphs, max matching = min vertex cover; max independent set = $|V| - \text{max matching}$.
- Project selection / max closure:** node weights w_v . $S \rightarrow v$ cap w_v if $w_v > 0$, $v \rightarrow T$ cap $-w_v$ if $w_v < 0$; for each dependency “ u needs v ” (edge $u \rightarrow v$) add $u \rightarrow v$ cap ∞ . Best = $\sum_{w_v > 0} w_v - \text{mincut}$; chosen set is the S side. Projects needing machines: project weight p_i , machine weight $-q_j$. Selecting an edge needs both endpoints: treat edges as projects depending on 2 nodes.
- Chains of choices:** to pick a value $h \in [0, H]$ per item, make nodes (i, h) with $(i, h) \rightarrow (i, h+1)$ cap ∞ (monotone), then cut costs become capacities on the chain.
- Min path cover in DAG (vertex-disjoint):** split v into v_L, v_R ; edge $u \rightarrow v$ becomes $u_L - v_R$. Answer = $|V| - \text{max matching}$. Not vertex-disjoint: use the transitive closure first.
- Max weight independent set, bipartite, $w \geq 0$:** $S \rightarrow l$ cap w_l , $r \rightarrow T$ cap w_r , $l \rightarrow r$ cap ∞ . Answer = $\sum w - \text{mincut}$.

5.2 Flow with lower bounds $[L, R]$

Edge $u \rightarrow v$ with bounds $[L, R]$: add $u \rightarrow v$ cap $R - L$, $u \rightarrow T'$ cap L , $S' \rightarrow v$ cap L (S', T' new). Add $T \rightarrow S$ cap ∞ . Feasible iff $\text{maxFlow}(S', T') = \sum L$. Max flow: on the *same* DinicFlow (no reset, keep $T \rightarrow S$) the value of $\text{maxFlow}(S, T)$ is the answer. Min flow: f on $T \rightarrow S$, remove it, answer $f - \text{maxFlow}(T, S)$. Assumes no edges into S or out of T . Circulation (no S, T): only the feasibility step.

DinicFlow.h

Description: Dinic max flow with capacity scaling. Nodes in $[0, n]$ (0- or 1-indexed). `addEdge(u, v, c, rc)` adds $u \rightarrow v$ cap c and $v \rightarrow u$ cap rc ($rc = c$ for undirected). `maxFlow` does NOT reset flow: a second call continues in the residual graph. After `maxFlow(s, t)`, `leftOfMinCut(v) = v` is on the s side of a min cut. Flow on an edge $e = \text{adj}[u][i]$: `e.flow()`.

Time: $\mathcal{O}(VE \log U)$, $\mathcal{O}(E\sqrt{V})$ unit caps / bipartite.

```
struct DinicFlow {
    struct E {
        int to, rev; ll c, oc;
        ll flow() { return max(oc - c, 0LL); }
    };
    vi lvl, ptr, q;
    vector<vector<E>> adj;
    DinicFlow(int n) : lvl(n+1), ptr(n+1), q(n+1), adj(n+1) {}
    void addEdge(int u, int v, ll c, ll rc = 0) {
        adj[u].push_back({v, sz(adj[v]), c, c});
        adj[v].push_back({u, sz(adj[u]) - 1, rc, rc});
    }
    ll dfs(int v, int t, ll f) {
        if (v == t || !f) return f;
        for (int &i = ptr[v]; i < sz(adj[v]); i++) {
            E &e = adj[v][i];
            if (lvl[e.to] == lvl[v] + 1)
                if (ll p = dfs(e.to, t, min(f, e.c))) {
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
        return 0;
    }
}
```

```
11 maxFlow(int s, int t) {
11 flow = 0; q[0] = s;
    rep(L, 0, 31) do {
        lvl = ptr = vi(sz(q));
        int qi = 0, qe = lvl[s] = 1;
        while (qi < qe && !lvl[t]) {
            int v = q[qi++];
            for (E e : adj[v])
                if (!lvl[e.to] && e.c >> (30 - L))
                    q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
        }
        while (lvl[p = dfs(s, t, LLONG_MAX)] flow += p;
    } while (lvl[t]);
    return flow;
}
bool leftOfMinCut(int v) { return lvl[v] != 0; }
};
```

MaxFlowMinCost.h

Description: Min cost max flow with SPFA. Nodes in $[0, n]$. Negative costs allowed, negative cycles are not. getFlow(s, t) resets flow and returns {flow, cost}.

Time: $\mathcal{O}(F \cdot VE)$, F = number of augmenting paths.

```
struct MaxFlowMinCost {
    struct Edge { int from, to; ll capa, flow, cost; };
    vector<vi> adj; vector<Edge> E;
    vector<ll> dist; vi tr;
    const ll INF = LLONG_MAX / 4;

    MaxFlowMinCost(int n) : adj(n + 1), dist(n + 1), tr(n + 1) {}

    void addEdge(int u, int v, ll ca, ll co) {
        adj[u].push_back(sz(E)); E.push_back({u, v, ca, 0, co});
        adj[v].push_back(sz(E)); E.push_back({v, u, 0, 0, -co});
    }

    bool spfa(int s, int t) {
        fill(all(dist), INF);
        vector<bool> inq(sz(dist));
        queue<int> qu; dist[s] = 0; qu.push(s);
        while (sz(qu)) {
            int u = qu.front(); qu.pop(); inq[u] = 0;
            for (int id : adj[u]) {
                Edge &e = E[id];
                if (e.flow < e.capa && dist[e.to] > dist[u] + e.cost) {
                    dist[e.to] = dist[u] + e.cost; tr[e.to] = id;
                    if (!inq[e.to]) inq[e.to] = 1, qu.push(e.to);
                }
            }
        }
        return dist[t] < INF;
    }

    pair<ll, ll> getFlow(int s, int t) {
        for (auto &e : E) e.flow = 0;
        ll fl = 0, cost = 0;
        while (spfa(s, t)) {
            ll d = INF;
            for (int u = t; u != s; u = E[tr[u]].from)
                d = min(d, E[tr[u]].capa - E[tr[u]].flow);
            for (int u = t; u != s; u = E[tr[u]].from)
                E[tr[u]].flow += d, E[tr[u]].flow -= d;
            fl += d, cost += d * dist[t];
        }
        return {fl, cost};
    }
};
```

GraphFastMatching.h

Description: Hopcroft-Karp bipartite matching. Left 1..m, right 1..n. After solve(): cntMatching, matx[u] = right partner or 0.

Time: $\mathcal{O}(E\sqrt{V})$

```
struct HopcroftKarp {
    vector<vector<int>>> adj;
    vector<int> dist, matx, maty;
    int mNode, nNode, cntMatching;

    HopcroftKarp(int _m, int _n) {
        mNode = _m, nNode = _n;
        adj = vector<vector<int>>>(mNode + 1, vector<int>());
        dist = matx = vector<int>(mNode + 1);
        maty = vector<int>(nNode + 1);
    }

    inline void addEdge(int u, int v) { adj[u].push_back(v); }

    bool bfs(void) {
        queue<int> qu;
        for (int i = 1; i <= mNode; ++i) {
            dist[i] = 1e9+7;
            if(!matx[i]) { dist[i] = 0; qu.push(i); }
        }
        dist[0] = 1e9+7;
        while (sz(qu)) {
            int u(qu.front()); qu.pop();
            if(dist[u] >= dist[0]) continue;
            for (int v : adj[u]) {
                if(dist[maty[v]] < 1e9+7) continue;
                dist[maty[v]] = dist[u] + 1;
                qu.push(maty[v]);
            }
        }
        return (dist[0] < 1e9+7);
    }

    bool dfs(int u) {
        if(!u) return true;
        for (int v : adj[u]) {
            if(dist[maty[v]] == dist[u] + 1 && dfs(maty[v])) {
                matx[u] = v, maty[v] = u; return true;
            }
        }
        dist[u] = 1e9+7;
        return false;
    }

    void solve(void) {
        cntMatching = 0;
        while(bfs()) for (int i = 1; i <= mNode; ++i) if(!matx[i])
            cntMatching += dfs(i);
    }

    void show(void) {
        cout << cntMatching << '\n';
        for (int i = 1; i <= mNode; ++i) if(matx[i]) cout << i << '
            ' << matx[i] << '\n';
    }
};
```

Hungarian.h

Description: Min cost perfect matching on an $n \times n$ cost matrix, 1-indexed. Negative costs OK. Rectangular: pad with 0 rows/cols. Max cost: negate. Missing edges default to 1e18. matchX[i] = column of row i. hungarian() returns the total cost.

Time: $\mathcal{O}(N^3)$

```
struct Hungarian {
    vector<vector<ll>>> c; // matrix cost
    vector<ll> fx, fy, dist;
    vi matchX, matchY; // potentials | corresponding node
    vi trace, arg; // prev left vertex | argmin of dist
    queue<int> qu; // used for bfs

    int numNode; // assume that |L| = |R| = n
    int start; // current root of the tree
    int finish; // leaf node of augmenting path

    Hungarian(int _n) {
        numNode = _n;
        c.assign(numNode + 1, vector<ll>(numNode + 1, 1e18));
        fx = fy = dist = vector<ll>(numNode + 1);
        matchX = matchY = trace = arg = vi(numNode + 1);
    }

    inline void addEdge(int u, int v, ll _cost) { c[u][v] = min(c[u][v], _cost); }
    inline ll cost(int u, int v) const { return c[u][v] - fx[u] - fy[v]; }

    void initBFS(int root) {
        start = root;
        qu = queue<int>(); qu.push(start);
        for (int i = 1; i <= numNode; ++i) {
            trace[i] = 0, arg[i] = start;
            dist[i] = cost(start, i);
        }
    }

    int findPath(void) {
        while (sz(qu)) {
            int u(qu.front()); qu.pop();
            for (int v = 1; v <= numNode; ++v) {
                if(trace[v]) continue;
                ll w = cost(u, v);
                if(w == 0) {
                    trace[v] = u;
                    if(!matchY[v]) return v;
                    qu.push(matchY[v]);
                }
                if(dist[v] > w) dist[v] = w, arg[v] = u;
            }
        }
        return 0;
    }

    void enlarge(void) {
        for (int y = finish, next; y != 0; y = next) {
            int x = trace[y]; next = matchX[x];
            matchX[x] = y, matchY[y] = x;
        }
    }

    void update(void) {
        ll delta = LLONG_MAX;
        for (int i = 1; i <= numNode; ++i) if(!trace[i]) delta = min(delta, dist[i]);
        fx[start] += delta;
        for (int i = 1; i <= numNode; ++i) {
            if(trace[i]) {
                fx[matchY[i]] += delta, fy[i] -= delta;
            } else {
                dist[i] -= delta;
                if(dist[i] == 0) {
                    trace[i] = arg[i];
                    if(matchY[i] == 0) { finish = i; }
                }
            }
        }
    }
};
```

```
        else qu.push(matchY[i]);
    }
}
}
}

11 hungarian(void) {
    for (int i = 1; i <= numNode; ++i)
        fx[i] = *min_element(c[i].begin() + 1, c[i].end());
    for (int i = 1; i <= numNode; ++i) {
        initBFS(i);
        do {
            finish = findPath();
            if(!finish) update();
        } while(!finish);
        enlarge();
    }
    11 ans = 0;
    for (int i = 1; i <= numNode; ++i) ans += c[i][matchX[i]];
    return ans;
}
};
```

EdmondsBlossom.h

Description: Maximum matching in a general graph (Edmonds’ blossom), 0-indexed. solve() returns the matching size; match[v] is v’s partner or -1.
Usage: Blossom B(n); B.addEdge(u, v); int k = B.solve();
Time: $\mathcal{O}(V^3)$

7c06a0, 55 lines

```
struct Blossom {
    int n;
    vector<vector<int>>> g;
    vector<int> match, p, base, q;
    vector<char> used, inBlossom;
    Blossom(int n) : n(n), g(n), match(n, -1), p(n), base(n), q(n),
        used(n), inBlossom(n) {}
    void addEdge(int u, int v) { g[u].push_back(v); g[v].push_back(u); }
    int lca(int a, int b) {
        vector<char> mark(n, 0);
        while (true) { a = base[a]; mark[a] = 1; if (match[a] == -1) break; a = p[match[a]]; }
        while (true) { b = base[b]; if (mark[b]) return b; b = p[match[b]]; }
    }
    void markPath(int v, int b, int children) {
        for (; base[v] != b; v = p[match[v]]) {
            inBlossom[base[v]] = inBlossom[base[match[v]]] = 1;
            p[v] = children; children = match[v];
        }
    }
    int findPath(int root) {
        fill(used.begin(), used.end(), 0); fill(p.begin(), p.end(), -1);
        for (int i = 0; i < n; i++) base[i] = i;
        used[root] = 1; int qh = 0, qt = 0; q[qt++] = root;
        while (qh < qt) {
            int v = q[qh++];
            for (int to : g[v]) {
                if (base[v] == base[to] || match[v] == to) continue;
                if (to == root || (match[to] != -1 && p[match[to]] != -1)) { // exist cycle with odd vertices -> have blossom
                    int curbase = lca(v, to);
                    fill(inBlossom.begin(), inBlossom.end(), 0);
                    markPath(v, curbase, to); markPath(to, curbase, v);
                    for (int i = 0; i < n; i++) if (inBlossom[base[i]]) {
                        base[i] = curbase;
                        if (!used[i]) { used[i] = 1; q[qt++] = i; }
                    }
                }
            }
        }
    }
};
```

```
    }
    } else if (p[to] == -1) {
        p[to] = v;
        if (match[to] == -1) return to; // augment path
        to = match[to]; used[to] = 1; q[qt++] = to;
    }
}
}
return -1;
}
int solve() {
    for (int i = 0; i < n; i++) if (match[i] == -1) // greedy
        init
        for (int to : g[i]) if (match[to] == -1) { match[i] = to;
            match[to] = i; break; }
    for (int i = 0; i < n; i++) if (match[i] == -1) {
        int v = findPath(i);
        while (v != -1) { int pv = p[v], ppv = match[pv]; match[v] = pv; match[pv] = v; v = ppv; }
    }
    int res = 0;
    for (int i = 0; i < n; i++) if (match[i] > i) res++;
    return res;
}
};
```

StoerWagner.h

Description: Global min cut of an undirected graph given as a weighted adjacency matrix. Returns {cut weight, vertices of one side}. For $n = 1$ returns {INF, {}}.
Time: $\mathcal{O}(V^3)$

76fb20, 28 lines

```
pair<ll, vi> stoerWagner(vector<vector<ll>>> w) {
    int n = sz(w);
    vector<vi> grp(n); // original vertices merged into i
    rep(i, 0, n) grp[i] = {i};
    vi id(n); iota(all(id), 0); // alive vertices
    ll best = LLONG_MAX; vi bestSet;
    while (sz(id) > 1) {
        int k = sz(id), prev = -1, last = -1;
        vector<ll> d(k); vector<char> in(k);
        rep(step, 0, k) {
            int sel = -1;
            rep(i, 0, k) if (!in[i] && (sel < 0 || d[i] > d[sel]))
                sel = i;
            in[sel] = 1; prev = last; last = sel;
            rep(i, 0, k) if (!in[i]) d[i] += w[id[sel]][id[i]];
        }
        if (d[last] < best)
            best = d[last], bestSet = grp[id[last]];
        int a = id[prev], b = id[last]; // merge b into a
        rep(i, 0, k) {
            w[a][id[i]] += w[b][id[i]];
            w[id[i]][a] = w[a][id[i]];
        }
        grp[a].insert(grp[a].end(), all(grp[b]));
        id.erase(id.begin() + last);
    }
    return {best, bestSet};
}
```

GomoryHuTree.h

Description: Gomory-Hu tree of an undirected graph with nodes $0..n - 1$ and edges $\{u, v, w\}$. Returns $n - 1$ tree edges $\{i, par, w\}$; min cut(u, v) = min weight on the tree path $u-v$.
Time: $n - 1$ max flow calls, $\mathcal{O}(V^3E)$.

"DinicFlow.h" 7cdf87, 12 lines

```
typedef array<ll, 3> Edge;
vector<Edge> gomoryHu(int n, const vector<Edge>& ed) {
```

```
vi par(n, 0); vector<Edge> tree;
rep(i, 1, n) {
    DinicFlow D(n);
    for (auto [u, v, w] : ed) D.addEdge(u, v, w, w);
    tree.push_back({i, par[i], D.maxFlow(i, par[i])});
    rep(j, i + 1, n)
        if (D.leftOfMinCut(j) && par[j] == par[i]) par[j] = i;
}
return tree;
}
```

Data structure (6)

ZKWSegTree.h

Description: Bottom-up (zkw) segment tree, 0-indexed. Point assign, query over $[l, r)$. f may be any associative function (need not be commutative); unit is its identity.
Usage: ZKW st(n); st.update(i, v); st.query(l, r + 1);
ZKW st(a) builds from a vector in $\mathcal{O}(n)$.
Time: $\mathcal{O}(\log N)$

07c12a, 24 lines

```
struct ZKW {
    typedef ll T;
    static constexpr T unit = 0;
    T f(T a, T b) { return a + b; } // (any associative fn)
    vector<T> s; int n;
    ZKW(int n = 0, T def = unit) : s(2 * n, def), n(n) {
        for (int i = n; --i > 0;) s[i] = f(s[2 * i], s[2 * i + 1]);
    }
    ZKW(const vector<T>& a) : ZKW(sz(a)) {
        copy(all(a), s.begin() + n);
        for (int i = n; --i > 0;) s[i] = f(s[2 * i], s[2 * i + 1]);
    }
    void update(int p, T v) {
        for (s[p += n] = v; p /= 2;) s[p] = f(s[2 * p], s[2 * p + 1]);
    }
    T query(int l, int r) { // [l, r)
        T ra = unit, rb = unit;
        for (l += n, r += n; l < r; l /= 2, r /= 2) {
            if (l & 1) ra = f(ra, s[l++]);
            if (r & 1) rb = f(s[--r], rb);
        }
        return f(ra, rb);
    }
};
```

LiChaoTreePtr.h

Description: Li Chao tree for MAXIMUM, dynamic nodes. update(L, R, u, v, line) adds line on x in $[u, v]$ (use $u=L, v=R$ for a full line); query(L, R, x). L, R = domain. For minimum insert (-a, -b) and negate the answer. Answers below -INF are clamped to -INF.
Time: $\mathcal{O}(\log C)$ per full line, $\mathcal{O}(\log^2 C)$ per segment.

ff5ae4, 64 lines

```
const ll INF = (ll)1e18;

struct LiLine {
    ll a, b;

    LiLine(ll a = 0, ll b = -INF) : a(a), b(b) {}

    inline ll operator () (ll x) const { return a * x + b; }
};

struct LiChao {
    LiLine value;
    LiChao* left;
    LiChao* rig;
```

```
LiChao(void) : value(LiLine()), lef(nullptr), rig(nullptr) {}

void update(int l, int r, int u, int v, const LiLine& LINE) {
    if (l > r or u > v or u > r or l > v)
        return;

    if (u <= l and r <= v) {
        LiLine current = value, other = LINE;
        if (current(l) > other(l))
            swap(current, other);

        if (current(r) <= other(r))
            value = other;
        else {
            if (l == r)
                return;
            int m = (l + r) >> 1;
            if (current(m) > other(m)) {
                value = current;
                lef = lef ? lef : new LiChao();
                lef->update(l, m, u, v, other);
            } else {
                value = other;
                rig = rig ? rig : new LiChao();
                rig->update(m + 1, r, u, v, current);
            }
        }
        return;
    }

    int m = (l + r) >> 1;
    lef = lef ? lef : new LiChao();
    rig = rig ? rig : new LiChao();

    lef->update(l, m, u, v, LINE);
    rig->update(m + 1, r, u, v, LINE);
}

ll query(int l, int r, int x) {
    if (l > r or x > r or l > x)
        return -INF;
    ll ans = value(x);

    int m = (l + r) >> 1;
    ans = max(ans, lef ? lef->query(l, m, x) : -INF);
    ans = max(ans, rig ? rig->query(m + 1, r, x) : -INF);
    return ans;
}

};
```

DynamicConvexHull.h
Description: Add lines $y = kx + m$, query MAXIMUM at x . For minimum, add $(-k, -m)$ and negate the query result.
Usage: For doubles, use $\text{inf} = 1/.0$, $\text{div}(a,b) = a/b$
Time: $\mathcal{O}(\log N)$

```
70d773, 31 lines
struct Line {
    ll k, m;
    mutable ll p;
    bool operator < (const Line& o) const { return k < o.k; }
    bool operator < (const ll &x) const { return p < x; }
};

struct DynamicHull : multiset<Line, less<>> {
    const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { return a / b - ((a ^ b) < 0 && a % b); }

    bool bad(iterator x, iterator y) {
```

```
if(y == end()) { x->p = inf; return false; }
if(x->k == y->k) x->p = x->m > y->m ? inf : -inf;
else x->p = div(y->m - x->m, x->k - y->k);
return x->p >= y->p;
}

void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (bad(y, z)) z = erase(z);
    if(x != begin() && bad(--x, y)) bad(x, y = erase(y));
    while((y = x) != begin() && (--x)->p >= y->p) bad(x, erase(y));
}

ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
}

};
```

SplayTree.h
Description: Implicit splay tree over positions $1..n$ Arr (node $i = \text{element } i$). $\text{root} = \text{buildSplay}(1, n\text{Arr})$; $\text{split}(T, A, B, c)$: $A = \text{first } c \text{ elements}$, $B = \text{rest}$; $\text{join}(A, B)$ concatenates. Only sizes are maintained: add lazy tags / sums in $\text{update}()$ and push them before descending in $\text{locate}()$ if needed.
Time: $\mathcal{O}(\log N)$ amortized.

```
1b725f, 77 lines
namespace SplayTree {
    const int MAXN = 200005;
    struct TNode {
        int sz, p, L, R;
        TNode() { sz = p = L = R = 0; }
    } node[MAXN];

    int root, nArr;

    inline void update(const int &x) {
        node[x].sz = 1 + node[node[x].L].sz + node[node[x].R].sz;
    }

    int buildSplay(int l, int r, int pa = 0) {
        if(l > r) return 0;
        int mid = (l + r) >> 1;
        node[mid].p = pa;
        node[mid].L = buildSplay(l, mid - 1, mid);
        node[mid].R = buildSplay(mid + 1, r, mid);
        update(mid);
        return mid;
    }

    inline void setChild(int pa, int child, bool isRight) {
        node[child].p = pa;
        (isRight ? node[pa].R : node[pa].L) = child;
    }

    inline void upTree(int x) {
        int y = node[x].p, z = node[y].p;
        if(x == node[y].R) {
            int b = node[x].L;
            setChild(y, b, 1), setChild(x, y, 0);
        } else {
            int b = node[x].R;
            setChild(y, b, 0), setChild(x, y, 1);
        }
        setChild(z, x, (node[z].R == y));
        update(y), update(x);
    }
}
```

```
void splay(int x) {
    while(1) {
        if(node[x].p == 0) break;
        int y = node[x].p, z = node[y].p;
        if(z > 0) {
            if((y == node[z].R) == (x == node[y].R)) { upTree(y); }
            else upTree(x);
        }
        upTree(x);
    }
}

int locate(int root, int c) {
    int x(root);
    while(1) {
        int s = node[node[x].L].sz;
        if(s + 1 == c) return x;
        if(s + 1 > c) { x = node[x].L; }
        else { c -= s + 1; x = node[x].R; }
    }
}

void split(int T, int &A, int &B, int c) {
    if(!c) { A = 0, B = T; return; }
    int x = locate(T, c);
    splay(x); A = x, B = node[A].R;
    node[A].R = node[B].p = 0; update(A);
}

int join(int A, int B) {
    if(A == 0) return B;
    while(node[A].R) A = node[A].R;
    splay(A), setChild(A, B, 1), update(A);
    return A;
}
}
```

OrderTree.h
Description: Set with order statistics. $\text{find_by_order}(k)$ returns the k -th element (0-indexed), $\text{order_of_key}(x)$ the number of elements $< x$. For a multiset use $\text{pair}<T, \text{int}>$.
Time: $\mathcal{O}(\log N)$
<bits/extc++.h> 819d08, 13 lines
using namespace __gnu_pbds;
template<class T>
using Tree = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;

```
void example() {
    Tree<int> t, t2; t.insert(8);
    auto it = t.insert(10).first;
    assert(it == t.lower_bound(9));
    assert(t.order_of_key(10) == 1);
    assert(t.order_of_key(11) == 2);
    assert(*t.find_by_order(0) == 8);
    t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}
```

HashMap.h
Description: Hash map 3x faster than unordered_map, with an anti-hack hash. Use like unordered_map (h[k], h.find).
<bits/extc++.h> 0c30cd, 6 lines
// To use most bits rather than just the lowest ones :
struct chash { // large odd number for C
 const uint64_t C = 11(4e18 * acos(0)) | 71;
 ll operator()(ll x) const { return __builtin_bswap64(x*C); }
};
__gnu_pbds::gp_hash_table<ll, int, chash> h({}, {}, {}, {}, {1<<16});

BitsetTree.h

Description: Set of integers in $[0, 64^{LAYER})$ (262144 for LAYER=3). update(x) toggles x. walk_forward(x) = smallest element $\geq x$, walk_backward(x) = largest element $< x$ (-1 if none). Call init() first. **Time:** $\mathcal{O}(LAYER)$ per operation.

3c903a, 76 lines

```

typedef unsigned long long ull;
struct BitsetTree {
    static const int LAYER = 3, BLOCK = 64;
    ull a[1 + 64 + 64 * 64]; // sum of 64^i, i < LAYER
    int layer_start[LAYER];

    void update(int x) {
        int id(x >> 6), prev(layer_start[LAYER - 1] + id);
        a[prev] ^= (1ULL << (x & (BLOCK - 1)));
        for (int i = LAYER - 2; i >= 0; --i) {
            x >>= 6, id >>= 6;
            int digit(x & (BLOCK - 1)), cur(layer_start[i] + id);
            if((a[prev] > 0) != (a[cur] >> digit & 1)) a[cur] ^= 1ULL
                << digit;
            prev = cur;
        }
    }

    inline int findR(ull mask, int i) {
        if(i >= BLOCK || !(mask &= ~0ULL << i)) return -1;
        return __builtin_ctzll(mask);
    }

    int walk_forward(int x) {
        int id(x >> 6), pos = findR(a[layer_start[LAYER - 1] + id],
            x & (BLOCK - 1));
        if(pos != -1) return x - (x & (BLOCK - 1)) + pos;
        for (int i = LAYER - 2; i >= 0; --i) {
            id >>= 6, x >>= 6;
            int cur(layer_start[i] + id), digit = (x & (BLOCK - 1)) +
                1;
            int pos = findR(a[cur], digit);
            if(pos != -1) {
                id = (id << 6) + pos;
                for (int j = i + 1; j < LAYER; ++j) {
                    int digit = __builtin_ctzll(a[layer_start[j] + id]);
                    id = (id << 6) + digit;
                }
                return id;
            }
        }
        return -1;
    }

    int findL(ull mask, int i) {
        if(i < BLOCK) mask &= (1ULL << i) - 1;
        if(!mask) return -1;
        return 63 - __builtin_clzll(mask);
    }

    int walk_backward(int x) {
        int id(x >> 6), pos = findL(a[layer_start[LAYER - 1] + id],
            x & (BLOCK - 1));
        if(pos != -1) return x - (x & (BLOCK - 1)) + pos;
        for (int i = LAYER - 2; i >= 0; --i) {
            id >>= 6, x >>= 6;
            int cur(layer_start[i] + id), digit = (x & (BLOCK - 1));
            int pos = findL(a[cur], digit);
            if(pos != -1) {
                id = (id << 6) + pos;
                for (int j = i + 1; j < LAYER; ++j) {
                    int digit = 63 - __builtin_clzll(a[layer_start[j] +
                        id]);
                }
            }
        }
    }
};

```

```

        id = (id << 6) + digit;
    }
    return id;
}

}
return -1;
}

void init(void) {
    memset(a, 0, sizeof(a));
    int cur(0), sum(1);
    for (int i = 0; i < LAYER; ++i) {
        layer_start[i] = cur;
        cur += sum, sum *= BLOCK;
    }
}

} bst;

```

ManhattanSpanningTree.h

Description: Manhattan MST candidate edges. using namespace ManhattanSpanningTree; fill p[1..nArr] (x, y, id), call buildEdge(edges), then run Kruskal on the $\leq 4N$ returned edges. **Time:** $\mathcal{O}(N \log N)$

cb3912, 57 lines

```

namespace ManhattanSpanningTree {
    const int MAXN = 200005;
    struct MPoint { ll x, y; int xy, id; } p[MAXN];
    struct Edge {
        int u, v; ll w;
        Edge(int a = 0, int b = 0, ll c = 0) : u(a), v(b), w(c) {}
    };
    int nArr;
    typedef pair<ll, int> pli;
    vector<ll> idx;
    pli fen[MAXN];
    int nTree;

    void modify(int i, pli x) {
        for (; i > 0; i -= i & -i) fen[i] = min(fen[i], x);
    }

    pli get(int i) {
        pli res = {LLONG_MAX, -1};
        for (; i <= nTree; i += i & -i) res = min(res, fen[i]);
        return res;
    }

    void buildSpan(vector<Edge> &edges) {
        sort(p + 1, p + nArr + 1, [](auto &a, auto &b) {
            return pair(a.x, a.y) < pair(b.x, b.y); });
        idx.clear();
        rep(i, 1, nArr + 1) idx.push_back(p[i].y - p[i].x);
        sort(all(idx));
        idx.erase(unique(all(idx)), idx.end());
        nTree = sz(idx);
        for (int i = 1; i <= nTree; ++i) fen[i] = {LLONG_MAX, -1};
        for (int i = 1; i <= nArr; ++i) p[i].xy = int(
            upper_bound(all(idx), p[i].y - p[i].x) - idx.begin());
        for (int i = nArr; i > 0; --i) {
            pli v = get(p[i].xy);
            if(v.second != -1) { MPoint &q = p[v.second];
                edges.push_back(Edge(p[i].id, q.id,
                    abs(p[i].x - q.x) + abs(p[i].y - q.y))); }
            modify(p[i].xy, pli(p[i].x + p[i].y, i));
        }
    }

    void buildEdge(vector<Edge> &edges) {

```

```

        edges.clear();
        for (int loop = 0; loop < 4; ++loop) {
            buildSpan(edges);
            for (int i = 1; i <= nArr; ++i) swap(p[i].x, p[i].y);
            buildSpan(edges);
            for (int i = 1; i <= nArr; ++i) {
                swap(p[i].x, p[i].y);
                if(loop & 1) p[i].y = -p[i].y;
                else p[i].x = -p[i].x;
            }
        }
    }
}

```

PersistentIT.h

Description: Persistent segment tree of counts over positions 1..n. init(n); root 0 is the empty version; newRoot = update(oldRoot, pos, val) adds val at pos; queryCnt(root, u, v) sums [u, v]. k-th smallest in a range: walk roots r[R] and r[L-1] together. **Time:** $\mathcal{O}(\log N)$ per operation, $\mathcal{O}(\log N)$ new nodes.

25d166, 42 lines

```

namespace PersistentSeg {
    const int MAXN = 200005;
    struct SegNode {
        int cnt, L, R;
    } seg[50 * MAXN];

    int nNode, nTree;

    void init(int _n) { nNode = _n, nTree = 0; }

    int update(int oldID, int l, int r, int pos, int val) {
        if(l == r) {
            seg[++nTree] = seg[oldID], seg[nTree].cnt += val;
            return nTree;
        }
        int cur(++nTree), mid = (l + r) >> 1;
        seg[cur] = seg[oldID];
        if(pos <= mid) {
            seg[cur].L = update(seg[oldID].L, l, mid, pos, val);
        } else {
            seg[cur].R = update(seg[oldID].R, mid + 1, r, pos, val);
        }
        seg[cur].cnt = seg[seg[cur].L].cnt + seg[seg[cur].R].cnt;
        return cur;
    }

    int update(int oldID, int pos, int val) {
        return update(oldID, 1, nNode, pos, val);
    }

    int queryCnt(int id, int l, int r, int u, int v) {
        if(u <= l && r <= v) return seg[id].cnt;
        int res = 0, mid = (l + r) >> 1;
        if(mid >= u) res += queryCnt(seg[id].L, l, mid, u, v);
        if(mid + 1 <= v) res += queryCnt(seg[id].R, mid + 1, r, u,
            v);
        return res;
    }

    int queryCnt(int id, int u, int v) {
        return queryCnt(id, 1, nNode, u, v);
    }
};

```

BipartiteDsu.h

Description: DSU keeping parity to the root. add(a,b) says a and b have different colours; ok(v) tells whether v's component is still bipartite. par(v) = colour of v relative to its root. Nodes 0..n-1.

Time: $\mathcal{O}(\alpha(N))$ amortized.

ae2912, 20 lines

```

struct BipDSU {
    vi p, d, rk, bip;
    BipDSU(int n) : p(n), d(n), rk(n), bip(n, 1) {
        iota(all(p), 0);
    }
    int find(int v) { // also sets d[v] = parity to root
        if (p[v] == v) return v;
        int r = find(p[v]);
        d[v] ^= d[p[v]]; return p[v] = r;
    }
    void add(int a, int b) {
        int x = find(a), y = find(b), w = d[a] ^ d[b] ^ 1;
        if (x == y) { if (w) bip[x] = 0; return; }
        if (rk[x] < rk[y]) swap(x, y);
        p[y] = x, d[y] = w, bip[x] ^= bip[y];
        rk[x] += rk[x] == rk[y];
    }
    bool ok(int v) { return bip[find(v)]; }
    int par(int v) { find(v); return d[v]; }
};

```

Graph (7)

TwoSat.h

Description: 2-SAT via Kosaraju. Variables are 1..n (index 0 is ignored). add_disjunction(u, nu, v, nv) adds clause (u xor nu) or (v xor nv), i.e. nu = 1 negates u.

Time: $\mathcal{O}(n + m)$

7b1120, 58 lines

```

class TWO_SAT {
private:
    int n;
    vector<vector<int>> forward_edge, back_edge;
    vector<bool> used;
    vector<int> order, comp;

    void dfs_first(int u) {
        used[u] = true;
        for (auto v : forward_edge[u])
            if (not used[v]) dfs_first(v);
        order.push_back(u);
    }

    void dfs_second(int u, int turn) {
        comp[u] = turn;
        for (auto v : back_edge[u])
            if (comp[v] == -1) dfs_second(v, turn);
    }
public:
    TWO_SAT(int n = 0) : n(n) {
        used.assign(2 * n + 7, false);
        comp.assign(2 * n + 7, -1);
        forward_edge.resize(2 * n + 7);
        back_edge.resize(2 * n + 7);
    }

    void add_disjunction(int a, bool na, int b, bool nb) {
        a = (a << 1) ^ na;
        b = (b << 1) ^ nb;
        int neg_a = a ^ 1;
        int neg_b = b ^ 1;
        forward_edge[neg_a].push_back(b);
        forward_edge[neg_b].push_back(a);
        back_edge[a].push_back(neg_b);
        back_edge[b].push_back(neg_a);
    }
};

```

```

vector<bool>* find_solution(void) {
    vector<bool> *assignment = new vector<bool> (2 * n + 7);

    for (int i = 2; i <= 2 * n + 1; i++)
        if (not used[i]) dfs_first(i);

    for (int i = 1, turn = 0; i <= 2 * n; i++) {
        int u = order[2 * n - i];
        if (comp[u] == -1) dfs_second(u, ++turn);
    }

    for (int i = 2; i <= 2 * n; i += 2) {
        if (comp[i] == comp[i + 1])
            return nullptr;
        (*assignment)[i / 2] = comp[i] > comp[i + 1];
    }

    return assignment;
}

```

HLD.h

Description: Heavy-light decomposition, nodes 0..n-1. work(root) builds; st[u] = position in base array. apply_on_path(x, y, f) calls f(l, r, fromX) on position ranges [l, r] in path order x -> y. Subtree of u = [st[u], en[u]].

Time: $\mathcal{O}(\log^2 n)$ per path with a segment tree.

33f3cc, 115 lines

constexpr int max_log = 20; // $2^{20} > n$

```

struct Tree {
    int n, T;
    std::vector<int> heavy, head, st, en, lvl, sz, pv;
    mutable std::vector<int> visited;
    std::vector<std::vector<int>> g, up;

    Tree(int _n = 0) : n(_n) {
        heavy.assign(n, -1);
        head.assign(n, 0);
        st.assign(n, 0);
        en.assign(n, 0);
        lvl.assign(n, 0);
        sz.assign(n, 0);
        pv.assign(n, 0);
        g.assign(n, {});
        up.assign(max_log, {});
        visited.assign(n, 0);
    }

    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    int dfs(int u) {
        sz[u] = 1;
        heavy[u] = -1;
        int max_size = 0;
        for (int v : g[u]) if (v ^ pv[u]) {
            pv[v] = u;
            lvl[v] = lvl[u] + 1;
            dfs(v);
            if (max_size < sz[v]) {
                max_size = sz[v];
                heavy[u] = v;
            }
            sz[u] += sz[v];
        }
        return sz[u];
    }
};

```

```

}

void decompose(int u, int h) {
    st[u] = T++;
    head[u] = h;
    if (heavy[u] != -1)
        decompose(heavy[u], h);
    for (int v : g[u]) if (v != pv[u] and v != heavy[u])
        decompose(v, v);
    en[u] = T;
}

void work(int x = 0) {
    T = 0;
    pv[x] = x;
    lvl[x] = 0;
    dfs(x);
    decompose(x, x);
    up[0] = pv;
    for (int j = 1; j < max_log; j += 1) {
        up[j].resize(n);
        for (int i = 0; i < n; i += 1)
            up[j][i] = up[j - 1][up[j - 1][i]];
    }
}

bool IS_ANCESTOR(int u, int v) const {
    return st[u] <= st[v] and en[u] >= en[v];
}

int LCA(int u, int v) const {
    if (IS_ANCESTOR(u, v))
        return u;
    if (IS_ANCESTOR(v, u))
        return v;
    for (int j = max_log - 1; j >= 0; j -= 1) {
        if (not IS_ANCESTOR(up[j][u], v))
            u = up[j][u];
    }
    return pv[u];
}

void apply_on_path(int x, int y, const std::function<void (
    int, int, bool)>& f) const {
    int z = LCA(x, y);
    {
        int v = x;
        while (v != z) {
            if (lvl[head[v]] <= lvl[z]) {
                f(st[z] + 1, st[v], true);
                break;
            }
            f(st[head[v]], st[v], true);
            v = pv[head[v]];
        }
    }
    f(st[z], st[z], false);
    {
        int v = y;
        int cnt_visited = 0;
        while (v != z) {
            if (lvl[head[v]] <= lvl[z]) {
                f(st[z] + 1, st[v], false);
                break;
            }
            visited[cnt_visited++] = v;
            v = pv[head[v]];
        }
        for (int at = cnt_visited - 1; at >= 0; at--) {

```

```
        v = visited[at];
        f(st[head[v]], st[v], false);
    }
}
};
```

Centroid.h

Description: Find centroid of the current component of size n. Needs child[v] = subtree sizes computed from the component root (skipping del[] nodes), adj[] and del[] (removed centroids).
Time: $O(n)$

```
const int N = 5e5 + 5;
vi adj[N]; int child[N]; bool del[N];
```

```
int centroid(int u, int parent, int n) {
    for (int v : adj[u])
        if (v != parent && child[v] > n/2 && !del[v])
            return centroid(v, u, n);
    return u;
}
```

BlockCutTree.h

Description: Block-cut tree, vertices 1..n (num[0] is the timer). Globals: adj[], num[], low[], lastComp[], stack<int> st, numNode, numBCC, adjp[] (size numNode + #blocks). Block i is node numNode + i, adjacent to its vertices. Call tarjan(u) for each unvisited u; isolated vertices get no block.
Time: $O(V + E)$

```
const int N = 5e5 + 5;
vi adj[N], adjp[2 * N];
int num[N], low[N], lastComp[N], numNode, numBCC;
stack<int> st;
```

```
void tarjan(int u) {
    low[u] = num[u] = ++num[0];
    for (int it = 0; it < int(adj[u].size()); ++it) {
        int v(adj[u][it]);
        if (!num[v]) {
            st.push(u); tarjan(v);
            low[u] = min(low[u], low[v]);
            if (low[v] == num[u]) {
                lastComp[u] = ++numBCC;
                adjp[u].push_back(numNode + numBCC);
                do {
                    v = st.top(); st.pop();
                    if (lastComp[v] != numBCC) {
                        lastComp[v] = numBCC;
                        adjp[numNode + numBCC].push_back(v);
                    }
                } while (v != u);
            }
        } else { low[u] = min(low[u], num[v]); }
    }
    st.push(u);
}
```

OnlineBridgeCounting.h

Description: Maintains the number of bridges while edges are added online. Vertices 0..n-1; call init(n) first, then add_edge(a, b); the global bridges holds the current count.
Time: $O(\log n)$ amortized per edge.

```
vector<int> par, dsu_2ecc, dsu_cc, dsu_cc_size;
int bridges;
int lca_iteration;
vector<int> last_visit;
```

```
void init(int n) {
    par.resize(n);
    dsu_2ecc.resize(n);
    dsu_cc.resize(n);
    dsu_cc_size.resize(n);
    lca_iteration = 0;
    last_visit.assign(n, 0);
    for (int i=0; i<n; ++i) {
        dsu_2ecc[i] = i;
        dsu_cc[i] = i;
        dsu_cc_size[i] = 1;
        par[i] = -1;
    }
    bridges = 0;
}
```

```
int find_2ecc(int v) {
    if (v == -1)
        return -1;
    return dsu_2ecc[v] == v ? v : dsu_2ecc[v] = find_2ecc(
        dsu_2ecc[v]);
}
```

```
int find_cc(int v) {
    v = find_2ecc(v);
    return dsu_cc[v] == v ? v : dsu_cc[v] = find_cc(dsu_cc[v]);
}
```

```
void make_root(int v) {
    int root = v;
    int child = -1;
    while (v != -1) {
        int p = find_2ecc(par[v]);
        par[v] = child;
        dsu_cc[v] = root;
        child = v;
        v = p;
    }
    dsu_cc_size[root] = dsu_cc_size[child];
}
```

```
void merge_path (int a, int b) {
    ++lca_iteration;
    vector<int> path_a, path_b;
    int lca = -1;
    while (lca == -1) {
        if (a != -1) {
            a = find_2ecc(a);
            path_a.push_back(a);
            if (last_visit[a] == lca_iteration) {
                lca = a;
                break;
            }
            last_visit[a] = lca_iteration;
            a = par[a];
        }
        if (b != -1) {
            b = find_2ecc(b);
            path_b.push_back(b);
            if (last_visit[b] == lca_iteration) {
                lca = b;
                break;
            }
            last_visit[b] = lca_iteration;
            b = par[b];
        }
    }
}
```

```
for (int v : path_a) {
    dsu_2ecc[v] = lca;
    if (v == lca)
        break;
    --bridges;
}
for (int v : path_b) {
    dsu_2ecc[v] = lca;
    if (v == lca)
        break;
    --bridges;
}
}
```

```
void add_edge(int a, int b) {
    a = find_2ecc(a);
    b = find_2ecc(b);
    if (a == b)
        return;
```

```
int ca = find_cc(a);
int cb = find_cc(b);
```

```
if (ca != cb) {
    ++bridges;
    if (dsu_cc_size[ca] > dsu_cc_size[cb]) {
        swap(a, b);
        swap(ca, cb);
    }
    make_root(a);
    par[a] = dsu_cc[a] = b;
    dsu_cc_size[cb] += dsu_cc_size[a];
} else {
    merge_path(a, b);
}
}
```

CheckBridgeArticulation.h

Description: Bridges and articulation points, vertices 1..n (num[0] is the timer). Globals: adj[u] = {v, edge id}, num[], low[] zeroed; call tarjan(u, -1) for each unvisited u. Multi-edges OK (compares edge ids).
Time: $O(V + E)$

```
const int N = 5e5 + 5;
vector<pii> adj[N]; // {v, edge id}
int num[N], low[N];
```

```
void tarjan(int u, int p_id) {
    low[u] = num[u] = ++num[0];
    int numChild = 0;
    for (auto [v, id] : adj[u]) {
        if (!num[v]) {
            tarjan(v, id);
            low[u] = min(low[u], low[v]);
            ++numChild;
            if (low[v] > num[u]) {} // (u, v) is a bridge
            if (p_id != -1 && low[v] >= num[u]) {} // u is cut
        } else if (id != p_id) low[u] = min(low[u], num[v]);
    }
    if (p_id == -1 && numChild >= 2) {} // root u is cut
}
```

MaximalClique.h

Description: Maximum clique size via Bron-Kerbosch, n <= 62. g[i][j] is the 1-indexed adjacency matrix. For max independent set, run on the complement graph.
Time: $O(3^{n/3})$

```
const int N = 64;
```

```
int g[N][N];
int res;
ll edges[N];

void BronKerbosch(int n, ll R, ll P, ll X) { //  $O(3^n)$ 
    // each leaf is a maximal clique R
    if (P == 0ll && X == 0ll) {
        int t = __builtin_popcountll(R);
        res = max(res, t);
        return;
    }

    int u = 0;
    while (!(1LL << u) & (P | X)) u++;
    for (int v = 0; v < n; v++) {
        if (((1LL << v) & P) && !((1LL << v) & edges[u])) {
            BronKerbosch(n, R | (1LL << v), P & edges[v], X & edges[v]);
            P -= (1LL << v);
            X |= (1LL << v);
        }
    }
}

int max_clique(int n) {
    res = 0;
    for (int i = 1; i <= n; i++) {
        edges[i - 1] = 0;
        for (int j = 1; j <= n; j++) if (g[i][j]) edges[i - 1] |= (1ll << (j - 1));
    }
    BronKerbosch(n, 0, (1ll << n) - 1, 0);
    return res;
}
```

EulerTour.h
Description: Euler walk (Hierholzer). Undirected: add edge id to both endpoints; directed: only to adj[from]. Start at an odd vertex (directed: out - in = 1) if any. No existence check: the walk is valid iff its size is m + 1. Recursion depth up to m.
Time: $\mathcal{O}(V + E)$

```
const int N = 5e5 + 5, M = 5e5 + 5; // vertices, edges
struct Edge {
    int target, id;

    Edge(int _target, int _id): target(_target), id(_id) {}
};

vector<Edge> adj[N];
bool used_edge[M];

list<int> euler_walk(int u) {
    list<int> ans;
    ans.push_back(u);

    while (!adj[u].empty()) {
        int v = adj[u].back().target;
        int eid = adj[u].back().id;

        adj[u].pop_back();
        if (used_edge[eid]) continue;

        used_edge[eid] = true;

        u = v;
        ans.push_back(u);
    }
}
```

```
for (auto it = ans.begin(); it != ans.end(); ++it)
    if (!adj[*it].empty()) {
        auto t = euler_walk(*it);
        t.pop_back();
        ans.splice(it, t);
    }

    return ans;
}
```

String (8)

KMP.h
Description: Prefix function, 1-indexed: str[0] is a dummy char, lps[i] = longest proper border of str[1..i]. To find pattern P in T run on " " + P + "#" + T.
Time: $\mathcal{O}(N)$

```
vi kmp(const string &str) {
    int n = sz(str) - 1; vi lps(n + 1);
    for (int i = 2; i <= n; ++i) {
        int k = lps[i - 1];
        while(k > 0 && str[k + 1] != str[i]) k = lps[k];
        lps[i] = k + (str[k + 1] == str[i]);
    }
    return lps;
}
```

Manacher.h
Description: Manacher, 1-indexed. Manacher M(s); then pod[i]: s[i-k.i+k] is a palindrome for k <= pod[i]; pev[i]: s[i-k+1.i+k] is a palindrome for k <= pev[i] (even, centred between i and i+1).
Time: $\mathcal{O}(N)$

```
struct Manacher {
    int n; string S; vi pod, pev;
    Manacher(const string& s) : n(sz(s)), S(" " + s),
        pod(n + 2), pev(n + 2) { calc_pod(); calc_pev(); }

    void calc_pod() {
        int L = 1, R = 0;
        for(int i = 1; i <= n; i++) {
            if(i > R) pod[i] = 0;
            else pod[i] = min(R - i, pod[L + (R - i)]);
            while(i - pod[i] - 1 > 0 && i + pod[i] + 1 <= n && S[i - pod[i] - 1] == S[i + pod[i] + 1]) pod[i]++;

            if(i + pod[i] > R) {
                R = i + pod[i];
                L = i - pod[i];
            }
        }
    }

    void calc_pev() {
        int L = 1, R = 0;
        for(int i = 1; i < n; i++) {
            int j = i + 1;
            if(j > R) pev[i] = 0;
            else pev[i] = min(R - j + 1, pev[L + (R - j)]);
            while(i - pev[i] > 0 && j + pev[i] <= n && S[i - pev[i]] == S[j + pev[i]]) pev[i]++;

            if(i + pev[i] > R) {
                R = i + pev[i];
                L = j - pev[i];
            }
        }
    }
}
```

AhoCorasick.h
Description: Aho-Corasick over alphabet [A, A+K). insert() all patterns, then build(); count(s) = total number of pattern occurrences in s. After build, cnt[v] = patterns that are suffixes of node v, nxt is the full automaton, link the suffix link.
Time: $\mathcal{O}(K \cdot \sum |P|)$ build, $\mathcal{O}(|s|)$ count.

```
namespace AhoCorasick {
    const int K = 26; const char A = 'a';
    struct Node { int nxt[K] = {}, link = 0; ll cnt = 0; };
    vector<Node> T(1);
    int insert(const string &s) {
        int v = 0;
        for (char ch : s) {
            int c = ch - A;
            if (!T[v].nxt[c])
                T[v].nxt[c] = sz(T), T.emplace_back();
            v = T[v].nxt[c];
        }
        T[v].cnt++; return v;
    }
    void build() {
        queue<int> q; q.push(0);
        while (sz(q)) {
            int v = q.front(), u = T[v].link; q.pop();
            if (v) T[v].cnt += T[u].cnt;
            rep(c, 0, K) {
                int &w = T[v].nxt[c];
                if (w) T[w].link = v ? T[u].nxt[c] : 0, q.push(w);
                else w = T[u].nxt[c];
            }
        }
    }
    ll count(const string &s) {
        int v = 0; ll r = 0;
        for (char ch : s) v = T[v].nxt[ch - A], r += T[v].cnt;
        return r;
    }
}
```

Hash.h
Description: Polynomial hash with NMOD moduli. H h(s); h.get(l, r) is the hash of s[l, r) (0-indexed). Hashes of equal-length substrings are comparable with ==.
Time: $\mathcal{O}(N)$ build, $\mathcal{O}(NMOD)$ per query.

```
const int NMOD = 2, BASE = 131;
const ll MD[] = {(1ll)1e9 + 2277, (1ll)1e9 + 5277};
typedef array<ll, NMOD> HV;

struct H {
    vector<HV> h, pw;
    H(const string& s) : h(sz(s) + 1), pw(sz(s) + 1) {
        rep(j, 0, NMOD) h[0][j] = 0, pw[0][j] = 1;
        rep(i, 0, sz(s)) rep(j, 0, NMOD) {
            h[i+1][j] = (h[i][j] * BASE + (unsigned char)s[i])
                % MD[j];
            pw[i+1][j] = pw[i][j] * BASE % MD[j];
        }
    }
    HV get(int l, int r) {
        HV res;
        rep(j, 0, NMOD) res[j] = ((h[r][j] - h[l][j] * pw[r-l][j])
            % MD[j] + MD[j]) % MD[j];
        return res;
    }
}
```

```
};
```

SuffixArray.h

Description: 0-indexed suffix array. Set str, strLen = sz(str), call buildSA(). sa[i] = start of i-th smallest suffix, pos = inverse of sa, lcp[i] = LCP(sa[i], sa[i+1]). Needs str[strLen] to be a unique terminator (std::string gives a NUL char).

Time: $\mathcal{O}(N \log^2(N))$

4b97cd, 31 lines

```
namespace SuffixArray {
    const int MAXN = 500005;
    string str; int strLen;
    int sa[MAXN], pos[MAXN], lcp[MAXN], tmp[MAXN], gap;
    bool sufCmp(int i, int j) {
        if(pos[i] != pos[j]) return (pos[i] < pos[j]);
        return (max(i, j) + gap < strLen) ? (pos[i + gap] < pos[j + gap]) : (i > j);
    }

    void buildLCP(void) {
        int k(0);
        for(int i = 0; i < strLen; ++i) {
            if(pos[i] != strLen - 1) {
                for(int j = sa[pos[i] + 1]; str[i + k] == str[j + k]; ) ++k;
                lcp[pos[i]] = k, k -= bool(k);
            }
        }
    }

    void buildSA(void) {
        for(int i = 0; i < strLen; ++i) sa[i] = i, pos[i] = str[i];
        for(gap = 1; ; gap <= 1) {
            sort(sa, sa + strLen, sufCmp);
            for(int i = 0; i + 1 < strLen; ++i)
                tmp[i + 1] = tmp[i] + sufCmp(sa[i], sa[i + 1]);
            for(int i = 0; i < strLen; ++i) pos[sa[i]] = tmp[i];
            if(tmp[strLen - 1] == strLen - 1) break;
        }
        buildLCP();
    }
}
```

SuffixArrayDC3.h

Description: Suffix array by SA-IS (file name says DC3). SuffixArray S(s); positions are 1-indexed: S.sa[1..n] = suffix starts (1..n) in order, S.pos = inverse, S.lcp[i] = LCP(sa[i], sa[i+1]) for i < n.

Time: $\mathcal{O}(N)$

f33610, 101 lines

```
#define MASK(i) (1LL<<(i))
#define BIT(x,i) ((x)>>(i)&1)
#define tget(i) BIT(t[(i)>>3], (i) & 7)
#define tset(i, b) { if (b) t[(i)>>3] |= MASK((i) & 7); else t[(i)>>3] &= ~MASK((i) & 7); }
#define chr(i) (cs == sizeof(int) ? ((int *)s)[i] : ((unc *)s)[i])
#define isLMS(i) ((i) > 0 && tget(i) && !tget((i) - 1))
```

```
typedef unsigned char unc;
```

```
class SuffixArray {
public:
    int n; string str; unc *s; vi sa, lcp, pos;
```

```
    void getBuckets(unc s[], vector<int> &bkt, int n, int k,
        int cs, int end) {
        for(int i = 0; i <= k; ++i) bkt[i] = 0;
        for(int i = 0; i < n; ++i) ++bkt[chr(i)];
        int sum = 0;
```

```
        for(int i = 0; i <= k; ++i) {
            sum += bkt[i];
            bkt[i] = (end) ? sum : sum - bkt[i];
        }
    }
```

```
    void inducesal(vector<unc> &t, int sa[], unc s[], vector<
        int> &bkt, int n, int k, int cs, int end) {
        getBuckets(s, bkt, n, k, cs, end);
        for(int i = 0; i < n; ++i) {
            int j = sa[i] - 1;
            if(j >= 0 && !tget(j)) sa[bkt[chr(j)]]++ = j;
        }
    }
```

```
    void inducesas(vector<unc> &t, int sa[], unc s[], vector<
        int> &bkt, int n, int k, int cs, int end) {
        getBuckets(s, bkt, n, k, cs, end);
        for(int i = n - 1; i >= 0; --i) {
            int j = sa[i] - 1;
            if(j >= 0 && tget(j)) sa[--bkt[chr(j)]] = j;
        }
    }
```

```
    void buildSA(unc s[], int sa[], int n, int k, int cs) {
        vector<unc> t = vector<unc>(n / 8 + 1, 0);
        int j;
        tset(n - 2, 0); tset(n - 1, 1);
        for(int i = n - 3; i >= 0; --i) tset(i, (chr(i) < chr(i + 1)) || (chr(i) == chr(i + 1) && tget(i + 1)));
        vector<int> bkt = vector<int>(k + 1, 0);
        getBuckets(s, bkt, n, k, cs, true);
        for(int i = n - 1; i >= 0; --i) sa[i] = -1;
        for(int i = n - 1; i >= 0; --i) if(isLMS(i)) sa[--bkt[chr(i)]] = i;
        inducesal(t, sa, s, bkt, n, k, cs, false);
        inducesas(t, sa, s, bkt, n, k, cs, true);
        bkt.clear();
        int nl = 0;
        for(int i = 0; i < n; ++i) if(isLMS(sa[i])) sa[nl++] = sa[i];
        for(int i = nl; i < n; ++i) sa[i] = -1;
        int name = 0, prev = -1;
        for(int i = 0; i < nl; ++i) {
            int pos = sa[i];
            bool diff = false;
            for(int d = 0; d < n; ++d) {
                if(prev < 0 || chr(prev + d) != chr(pos + d) || tget(prev + d) != tget(pos + d)) {
                    diff = true;
                    break;
                } else if(d > 0 && (isLMS(prev + d) || isLMS(pos + d))) break;
            }
            if(diff) ++name, prev = pos;
            sa[nl + pos / 2] = name - 1;
        }
        j = n - 1;
        for(int i = n - 1; i >= nl; --i) if(sa[i] >= 0) sa[j--] = sa[i];
        int *sal = sa, *sl = sa + n - nl;
        if(name < nl) buildSA((unc*)sl, sal, nl, name - 1, sizeof(int));
        else for(int i = 0; i < nl; ++i) sal[sl[i]] = i;
        bkt.assign(k + 1, 0);
        getBuckets(s, bkt, n, k, cs, true);
        j = 0;
        for(int i = 0; i < n; ++i) if(isLMS(i)) sl[j++] = i;
        for(int i = 0; i < nl; ++i) sal[i] = sl[sal[i]];
```

```
        for(int i = nl; i < n; ++i) sa[i] = -1;
        for(int i = nl - 1; i >= 0; --i) j = sa[i], sa[i] = -1, sa[--bkt[chr(j)]] = j;
        inducesal(t, sa, s, bkt, n, k, cs, false);
        inducesas(t, sa, s, bkt, n, k, cs, true);
        bkt.clear(), t.clear();
    }
```

```
    void buildLCP(void) {
        for(int i = 1; i <= n; ++i) pos[sa[i]] = i;
        int k = 0; lcp[n] = 0;
        for(int i = 1; i <= n; ++i) if(pos[i] < n) {
            for(int j = sa[pos[i] + 1]; s[i + k - 1] == s[j + k - 1]; ) ++k;
            lcp[pos[i]] = k; k -= (k > 0);
        }
    }
```

```
    SuffixArray(const string& ss) : n(sz(ss)), str(ss),
        sa(n + 7), lcp(n + 7), pos(n + 7) {
        s = (unc*) str.c_str();
        if(!n) return;
        buildSA(s, sa.data(), n + 1, 256, sizeof(char));
        for(int i = 1; i <= n; ++i) ++sa[i];
        buildLCP();
    }
```

};

MinimalStringRotation.h

Description: Start index of the lexicographically minimal rotation of s (smallest such index on ties).

Time: $\mathcal{O}(N)$

d89251, 20 lines

```
int minmove(string s) {
    int n = s.length();
    int x, y, i, j, u, v; // x is the smallest string before
        string y
    for(x = 0, y = 1; y < n; ++y) {
        i = u = x;
        j = v = y;
        while(s[i] == s[j]) {
            ++u; ++v;
            if(++i == n) i = 0;
            if(++j == n) j = 0;
            if(i == x) break; // All strings are equal
        }
        if(s[i] <= s[j]) y = v;
        else {
            x = y;
            if(u > y) y = u;
        }
    }
    return x;
}
```

LyndonWord.h

Description: Duval: prints the Lyndon factorization of s (non-increasing sequence of Lyndon words).

Time: $\mathcal{O}(N)$

70ef25, 17 lines

```
void lyndon(string s) {
    int n = (int) s.length();
    int i = 0;
    while(i < n) {
        int j = i + 1, k = i;
        while(j < n && s[k] <= s[j]) {
            if(s[k] < s[j]) k = i;
            else ++k;
            ++j;
        }
```

```
    }
    while (i <= k) {
        cout << s.substr(i, j - k) << ' ';
        i += j - k;
    }
}
cout << endl;
}
```

Geometry (9)

9.1 Geometric primitives

Point.h
Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

```
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
    typedef Point P;
    T x, y;
    explicit Point(T x=0, T y=0) : x(x), y(y) {}
    bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
    bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
    P operator+(P p) const { return P(x+p.x, y+p.y); }
    P operator-(P p) const { return P(x-p.x, y-p.y); }
    P operator*(T d) const { return P(x*d, y*d); }
    P operator/(T d) const { return P(x/d, y/d); }
    T dot(P p) const { return x*p.x + y*p.y; }
    T cross(P p) const { return x*p.y - y*p.x; }
    T cross(P a, P b) const { return (a-*this).cross(b-*this); }
    T dist2() const { return x*x + y*y; }
    double dist() const { return sqrt((double)dist2()); }
    // angle to x-axis in interval [-pi, pi]
    double angle() const { return atan2(y, x); }
    P unit() const { return *this/dist(); } // makes dist()==1
    P perp() const { return P(-y, x); } // rotates +90 degrees
    P normal() const { return perp().unit(); }
    // returns point rotated 'a' radians ccw around the origin
    P rotate(double a) const {
        return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
    friend ostream& operator<<(ostream& os, P p) {
        return os << "(" << p.x << ", " << p.y << ")"; }
};
```

lineDistance.h
Description: Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.

```
"Point.h"
template<class P>
double lineDist(const P& a, const P& b, const P& p) {
    return (double) (b-a).cross(p-a) / (b-a).dist();
}
```

SegmentDistance.h
Description: Returns the shortest distance between point p and the line segment from point s to e.

```
Usage: Point<double> a, b(2,2), p(1,1);
bool onSegment = segDist(a,b,p) < 1e-10;
"Point.h"
typedef Point<double> P;
double segDist(P s, P e, P p) {
    if (s==e) return (p-s).dist();
    auto d = (e-s).dist2(), t = min(d,max(.0, (p-s).dot(e-s)));
    return ((p-s)*d-(e-s)*t).dist()/d;
}
```

SegmentIntersection.h
Description: If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.

```
Usage: vector<P> inter = segInter(s1,e1,s2,e2);
if (sz(inter)==1)
    cout << "segments intersect at " << inter[0] << endl;
"Point.h", "OnSegment.h"
template<class P> vector<P> segInter(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b),
        oc = a.cross(b, c), od = a.cross(b, d);
    // Checks if intersection is single non-endpoint point.
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa)};
    set<P> s;
    if (onSegment(c, d, a)) s.insert(a);
    if (onSegment(c, d, b)) s.insert(b);
    if (onSegment(a, b, c)) s.insert(c);
    if (onSegment(a, b, d)) s.insert(d);
    return {all(s)};
}
```

lineIntersection.h
Description: If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.

```
Usage: auto res = lineInter(s1,e1,s2,e2);
if (res.first == 1)
    cout << "intersection point at " << res.second << endl;
"Point.h"
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) // if parallel
        return {-(s1.cross(e1, s2) == 0), P(0, 0)};
    auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
    return {1, (s1 * p + e1 * q) / d};
}
```

sideOf.h
Description: Returns where p is as seen from s towards e. 1/0/-1 ⇔ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

```
Usage: bool left = sideOf(p1,p2,q)==1;
"Point.h"
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }

template<class P>
int sideOf(const P& s, const P& e, const P& p, double eps) {
    auto a = (e-s).cross(p-s);
    double l = (e-s).dist()*eps;
    return (a > l) - (a < -l);
}
```

OnSegment.h
Description: Returns true iff p lies on the line segment from s to e. Use (segDist(s,e,p) <= epsilon) instead when using Point<double>.

```
"Point.h"
template<class P> bool onSegment(P s, P e, P p) {
    return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

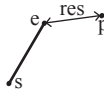
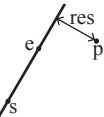
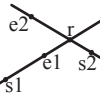
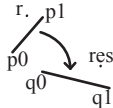
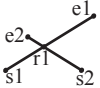
linearTransformation.h
Description: Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.

```
"Point.h"
typedef Point<double> P;
P linearTransformation(const P& p0, const P& p1,
    const P& q0, const P& q1, const P& r) {
    P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
    return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
}
```

Angle.h
Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.
Usage: vector<Angle> v = {w[0], w[0].t360() ...}; // sorted
int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }
// sweeps j such that (j-i) represents the number of positively oriented triangles with vertices at 0 and i

```
"Point.h"
struct Angle {
    int x, y;
    int t;
    Angle(int x, int y, int t=0) : x(x), y(y), t(t) {}
    Angle operator-(Angle b) const { return {x-b.x, y-b.y, t}; }
    int half() const {
        assert(x || y);
        return y < 0 || (y == 0 && x < 0);
    }
    Angle t90() const { return {-y, x, t + (half() && x >= 0)}; }
    Angle t180() const { return {-x, -y, t + half(); }
    Angle t360() const { return {x, y, t + 1}; }
};
bool operator<(Angle a, Angle b) {
    // add a.dist2() and b.dist2() to also compare distances
    return make_tuple(a.t, a.half(), a.y * (ll)b.x) <
        make_tuple(b.t, b.half(), a.x * (ll)b.y);
}
```

// Given two points, this calculates the smallest angle between // them, i.e., the angle that covers the defined line segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
 if (b < a) swap(a, b);
 return (b < a.t180() ?
 make_pair(a, b) : make_pair(b, a.t360()));
}



```
Angle operator+(Angle a, Angle b) { // point a + vector b
    Angle r(a.x + b.x, a.y + b.y, a.t);
    if (a.t180() < r) r.t--;
    return r.t180() < a ? r.t360() : r;
}
Angle angleDiff(Angle a, Angle b) { // angle b - angle a
    int tu = b.t - a.t; a.t = b.t;
    return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b < a)};
}
```

9.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

```
"Point.h" 84d6d3, 11 lines

typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,pair<P, P>* out) {
    if (a == b) { assert(r1 != r2); return false; }
    P vec = b - a;
    double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
        p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2;
    if (sum*sum < d2 || dif*dif > d2) return false;
    P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2);
    *out = {mid + per, mid - per};
    return true;
}
```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

```
"Point.h" b0153d, 13 lines

template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {
    P d = c2 - c1;
    double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
    if (d2 == 0 || h2 < 0) return {};
    vector<pair<P, P>> out;
    for (double sign : {-1, 1}) {
        P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
        out.push_back({c1 + v * r1, c2 + v * r2});
    }
    if (h2 == 0) out.pop_back();
    return out;
}
```

CirclePolygonIntersection.h

Description: Returns the area of the intersection of a circle with a ccw polygon. **Time:** $\mathcal{O}(n)$

```
"../../../../content/geometry/Point.h" 19add1, 19 lines

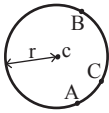
typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
    auto tri = [&](P p, P q) {
        auto r2 = r * r / 2;
        P d = q - p;
        auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2();
        auto det = a * a - b;
        if (det <= 0) return arg(p, q) * r2;
        auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det));
        if (t < 0 || 1 <= s) return arg(p, q) * r2;
        P u = p + d * s, v = q + d * (t-1);
```

```
        return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2;
    };
    auto sum = 0.0;
    rep(i,0,sz(ps))
        sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
    return sum;
}
```

circumcircle.h

Description:

The circumcircle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.



```
"Point.h" 1caa3a, 9 lines

typedef Point<double> P;
double ccRadius(const P& A, const P& B, const P& C) {
    return (B-A).dist()*(C-B).dist()*(A-C).dist()/
        abs((B-A).cross(C-A))/2;
}
P ccCenter(const P& A, const P& B, const P& C) {
    P b = C-A, c = B-A;
    return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}
```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points. **Time:** expected $\mathcal{O}(n)$

```
"circumcircle.h" 09dd0a, 17 lines

pair<P, double> mec(vector<P> ps) {
    shuffle(all(ps), mt19937(time(0)));
    P o = ps[0];
    double r = 0, EPS = 1 + 1e-8;
    rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) {
        o = ps[i], r = 0;
        rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) {
            o = (ps[i] + ps[j]) / 2;
            r = (o - ps[i]).dist();
            rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) {
                o = ccCenter(ps[i], ps[j], ps[k]);
                r = (o - ps[i]).dist();
            }
        }
    }
    return {o, r};
}
```

9.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

Usage: vector<P> v = {P{4,4}, P{1,2}, P{2,1}}; bool in = inPolygon(v, P{3, 3}, false); **Time:** $\mathcal{O}(n)$

```
"Point.h", "OnSegment.h", "SegmentDistance.h" 2bf504, 11 lines

template<class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
    int cnt = 0, n = sz(p);
    rep(i,0,n) {
        P q = p[(i + 1) % n];
        if (onSegment(p[i], q, a)) return !strict;
        //or: if (segDist(p[i], q, a) <= eps) return !strict;
        cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
    }
    return cnt;
}
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

```
"Point.h" f12300, 6 lines

template<class T>
T polygonArea2(vector<Point<T>>& v) {
    T a = v.back().cross(v[0]);
    rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
    return a;
}
```

PolygonCenter.h

Description: Returns the center of mass for a polygon. **Time:** $\mathcal{O}(n)$

```
"Point.h" 9706dc, 9 lines

typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
    P res(0, 0); double A = 0;
    for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
        res = res + (v[i] + v[j]) * v[j].cross(v[i]);
        A += v[j].cross(v[i]);
    }
    return res / A / 3;
}
```

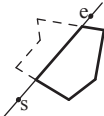
PolygonCut.h

Description: Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.

Usage: vector<P> p = ...; p = polygonCut(p, P(0,0), P(1,0));

```
"Point.h" d07181, 13 lines

typedef Point<double> P;
vector<P> polygonCut(const vector<P>& poly, P s, P e) {
    vector<P> res;
    rep(i,0,sz(poly)) {
        P cur = poly[i], prev = i ? poly[i-1] : poly.back();
        auto a = s.cross(e, cur), b = s.cross(e, prev);
        if ((a < 0) != (b < 0))
            res.push_back(cur + (prev - cur) * (a / (a - b)));
        if (a < 0)
            res.push_back(cur);
    }
    return res;
}
```



ConvexHull.h

Description: Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull. **Time:** $\mathcal{O}(n \log n)$

```
"Point.h" 310954, 13 lines

typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
    if (sz(pts) <= 1) return pts;
    sort(all(pts));
    vector<P> h(sz(pts)+1);
    int s = 0, t = 0;
    for (int it = 2; it--; s = --t, reverse(all(pts)))
        for (P p : pts) {
            while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
            h[t++] = p;
        }
    return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```



HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).

Time: $\mathcal{O}(n)$

```
"Point.h" c571b8, 12 lines

typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
    int n = sz(S), j = n < 2 ? 0 : 1;
    pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
    rep(i,0,j)
        for (; j = (j + 1) % n) {
            res = max(res, {{S[i] - S[j]}.dist2(), {S[i], S[j]}});
            if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
                break;
        }
    return res.second;
}
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.

Time: $\mathcal{O}(\log N)$

```
"Point.h", "sideOf.h", "OnSegment.h" 71446b, 14 lines

typedef Point<ll> P;

bool inHull(const vector<P>& l, P p, bool strict = true) {
    int a = 1, b = sz(l) - 1, r = !strict;
    if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
    if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
    if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p) <= -r)
        return false;
    while (abs(a - b) > 1) {
        int c = (a + b) / 2;
        (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
    }
    return sgn(l[a].cross(l[b], p)) < r;
}
```

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: $\bullet(-1, -1)$ if no collision, $\bullet(i, -1)$ if touching the corner i , $\bullet(i, i)$ if along side $(i, i + 1)$, $\bullet(i, j)$ if crossing sides $(i, i + 1)$ and $(j, j + 1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i, i + 1)$. The points are returned in the same order as the line hits the polygon. extrVertex returns the point of a hull with the max projection onto a line.

Time: $\mathcal{O}(\log n)$

```
"Point.h" 7cf45b, 39 lines

#define cmp(i, j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P> int extrVertex(vector<P>& poly, P dir) {
    int n = sz(poly), lo = 0, hi = n;
    if (extr(0)) return 0;
    while (lo + 1 < hi) {
        int m = (lo + hi) / 2;
        if (extr(m)) return m;
        int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
        (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
    }
    return lo;
}

#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
    int endA = extrVertex(poly, (a - b).perp());
```

```
int endB = extrVertex(poly, (b - a).perp());
if (cmpL(endA) < 0 || cmpL(endB) > 0)
    return {-1, -1};
array<int, 2> res;
rep(i,0,2) {
    int lo = endB, hi = endA, n = sz(poly);
    while ((lo + 1) % n != hi) {
        int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
        (cmpL(m) == cmpL(endB) ? lo : hi) = m;
    }
    res[i] = (lo + !cmpL(hi)) % n;
    swap(endA, endB);
}
if (res[0] == res[1]) return {res[0], -1};
if (!cmpL(res[0]) && !cmpL(res[1]))
    switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
        case 0: return {res[0], res[0]};
        case 2: return {res[1], res[1]};
    }
return res;
}
```

9.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.

Time: $\mathcal{O}(n \log n)$

```
"Point.h" ac41a6, 17 lines

typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
    assert(sz(v) > 1);
    set<P> S;
    sort(all(v), [](P a, P b) { return a.y < b.y; });
    pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
    int j = 0;
    for (P p : v) {
        P d(1 + (ll)sqrt(ret.first), 0);
        while (v[j].y <= p.y - d.x) S.erase(v[j++]);
        auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
        for (; lo != hi; ++lo)
            ret = min(ret, {(lo - p).dist2(), {lo, p}});
        S.insert(p);
    }
    return ret.second;
}
```

kdTree.h

Description: KD-tree (2d, can be extended to 3d)

```
"Point.h" bac5b0, 63 lines

typedef long long T;
typedef Point<T> P;
const T INF = numeric_limits<T>::max();

bool on_x(const P& a, const P& b) { return a.x < b.x; }
bool on_y(const P& a, const P& b) { return a.y < b.y; }

struct Node {
    P pt; // if this is a leaf, the single point in it
    T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds
    Node *first = 0, *second = 0;

    T distance(const P& p) { // min squared distance to a point
        T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x);
        T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y);
        return (P(x,y) - p).dist2();
    }

    Node(vector<P>&& vp) : pt(vp[0]) {
        for (P p : vp) {
```

```
x0 = min(x0, p.x); x1 = max(x1, p.x);
y0 = min(y0, p.y); y1 = max(y1, p.y);
}
if (vp.size() > 1) {
    // split on x if width >= height (not ideal...)
    sort(all(vp), x1 - x0 >= y1 - y0 ? on_x : on_y);
    // divide by taking half the array for each child (not
    // best performance with many duplicates in the middle)
    int half = sz(vp)/2;
    first = new Node({vp.begin(), vp.begin() + half});
    second = new Node({vp.begin() + half, vp.end()});
}
};

struct KDTree {
    Node* root;
    KDTree(const vector<P>& vp) : root(new Node({all(vp)})) {}

    pair<T, P> search(Node *node, const P& p) {
        if (!node->first) {
            // uncomment if we should not find the point itself:
            // if (p == node->pt) return {INF, P()};
            return make_pair((p - node->pt).dist2(), node->pt);
        }

        Node *f = node->first, *s = node->second;
        T bfirst = f->distance(p), bsec = s->distance(p);
        if (bfirst > bsec) swap(bsec, bfirst), swap(f, s);

        // search closest side first, other side if needed
        auto best = search(f, p);
        if (bsec < best.first)
            best = min(best, search(s, p));
        return best;
    }

    // find nearest point to a point, and its squared distance
    // (requires an arbitrary operator< for Point)
    pair<T, P> nearest(const P& p) {
        return search(root, p);
    }
};
```

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order $\{t[0][0], t[0][1], t[0][2], t[1][0], \dots\}$, all counter-clockwise.

Time: $\mathcal{O}(n \log n)$

```
"Point.h" eefdf5, 88 lines

typedef Point<ll> P;
typedef struct Quad* Q;
typedef __int128_t ll1; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX, LLONG_MAX); // not equal to any other point

struct Quad {
    Q rot, o; P p = arb; bool mark;
    P& F() { return r()->p; }
    Q& r() { return rot->rot; }
    Q prev() { return rot->o->rot; }
    Q next() { return r()->prev(); }
} *H;

bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
    ll1 p2 = p.dist2(), A = a.dist2()-p2,
        B = b.dist2()-p2, C = c.dist2()-p2;
    return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B > 0;
```

```

}
Q makeEdge(P orig, P dest) {
    Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};
    H = r->o; r->r()->r() = r;
    rep(i,0,4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r();
    r->p = orig; r->F() = dest;
    return r;
}
void splice(Q a, Q b) {
    swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
}
Q connect(Q a, Q b) {
    Q q = makeEdge(a->F(), b->p);
    splice(q, a->next());
    splice(q->r(), b);
    return q;
}

pair<Q,Q> rec(const vector<P>& s) {
    if (sz(s) <= 3) {
        Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
        if (sz(s) == 2) return { a, a->r() };
        splice(a->r(), b);
        auto side = s[0].cross(s[1], s[2]);
        Q c = side ? connect(b, a) : 0;
        return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
    }

#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
    Q A, B, ra, rb;
    int half = sz(s) / 2;
    tie(ra, A) = rec({all(s) - half});
    tie(B, rb) = rec({sz(s) - half + all(s)});
    while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
        (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
    Q base = connect(B->r(), A);
    if (A->p == ra->p) ra = base->r();
    if (B->p == rb->p) rb = base;

#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
    while (circ(e->dir->F(), H(base), e->F())) { \
        Q t = e->dir; \
        splice(e, e->prev()); \
        splice(e->r(), e->r()->prev()); \
        e->o = H; H = e; e = t; \
    }
    for (;;) {
        DEL(LC, base->r(), o); DEL(RC, base, prev());
        if (!valid(LC) && !valid(RC)) break;
        if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
            base = connect(RC, base->r());
        else
            base = connect(base->r(), LC->r());
    }
    return { ra, rb };
}

vector<P> triangulate(vector<P> pts) {
    sort(all(pts)); assert(unique(all(pts)) == pts.end());
    if (sz(pts) < 2) return {};
    Q e = rec(pts).first;
    vector<Q> q = {e};
    int qi = 0;
    while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
    q.push_back(c->r()); c = c->next(); } while (c != e); }
    ADD; pts.clear();
    while (qi < sz(q)) if (!(e = q[qi++])->mark) ADD;
```

```

    return pts;
}
```

9.5 3D

PolyhedronVolume.h

Description: Magic formula for the volume of a polyhedron. Faces should point outwards.

```

template<class V, class L>
double signedPolyVolume(const V& p, const L& trilst) {
    double v = 0;
    for (auto i : trilst) v += p[i.a].cross(p[i.b]).dot(p[i.c]);
    return v / 6;
}
```

Point3D.h

Description: Class to handle points in 3D space. T can be e.g. double or long long.

```

template<class T> struct Point3D {
    typedef Point3D P;
    typedef const P& R;
    T x, y, z;
    explicit Point3D(T x=0, T y=0, T z=0) : x(x), y(y), z(z) {}
    bool operator<(R p) const {
        return tie(x, y, z) < tie(p.x, p.y, p.z); }
    bool operator==(R p) const {
        return tie(x, y, z) == tie(p.x, p.y, p.z); }
    P operator+(R p) const { return P(x+p.x, y+p.y, z+p.z); }
    P operator-(R p) const { return P(x-p.x, y-p.y, z-p.z); }
    P operator*(T d) const { return P(x*d, y*d, z*d); }
    P operator/(T d) const { return P(x/d, y/d, z/d); }
    T dot(R p) const { return x*p.x + y*p.y + z*p.z; }
    P cross(R p) const {
        return P(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x);
    }
    T dist2() const { return x*x + y*y + z*z; }
    double dist() const { return sqrt((double)dist2()); }
    //Azimuthal angle (longitude) to x-axis in interval [-pi, pi]
    double phi() const { return atan2(y, x); }
    //Zenith angle (latitude) to the z-axis in interval [0, pi]
    double theta() const { return atan2(sqrt(x*x+y*y),z); }
    P unit() const { return *this/(T)dist(); } //makes dist()==1
    //returns unit vector normal to *this and p
    P normal(P p) const { return cross(p).unit(); }
    //returns point rotated 'angle' radians ccw around axis
    P rotate(double angle, P axis) const {
        double s = sin(angle), c = cos(angle); P u = axis.unit();
        return u.dot(u)*(1-c) + (*this)*c - cross(u)*s;
    }
};
```

3dHull.h

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards.

Time: $\mathcal{O}(n^2)$

```

"Point3D.h"
typedef Point3D<double> P3;

struct PR {
    void ins(int x) { (a == -1 ? a : b) = x; }
    void rem(int x) { (a == x ? a : b) = -1; }
    int cnt() { return (a != -1) + (b != -1); }
    int a, b;
};

struct F { P3 q; int a, b, c; };
```

```

vector<F> hull3d(const vector<P3>& A) {
    assert(sz(A) >= 4);
    vector<vector<PR>> E(sz(A), vector<PR>(sz(A), {-1, -1}));
#define E(x,y) E[f.x][f.y]
    vector<F> FS;
    auto mf = [&](int i, int j, int k, int l) {
        P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
        if (q.dot(A[l]) > q.dot(A[i]))
            q = q * -1;
        F f{q, i, j, k};
        E(a,b).ins(k); E(a,c).ins(j); E(b,c).ins(i);
        FS.push_back(f);
    };
    rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4)
        mf(i, j, k, 6 - i - j - k);

    rep(i,4,sz(A)) {
        rep(j,0,sz(FS)) {
            F f = FS[j];
            if (f.q.dot(A[i]) > f.q.dot(A[f.a])) {
                E(a,b).rem(f.c);
                E(a,c).rem(f.b);
                E(b,c).rem(f.a);
                swap(FS[j--], FS.back());
                FS.pop_back();
            }
        }
        int nw = sz(FS);
        rep(j,0,nw) {
            F f = FS[j];
#define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f.c);
            C(a, b, c); C(a, c, b); C(b, c, a);
        }
        for (F& it : FS) if ((A[it.b] - A[it.a]).cross(
            A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b);
        return FS;
    };
};
```

sphericalDistance.h

Description: Returns the shortest distance on the sphere with radius r radius between the points with azimuthal angles (longitude) f1 (ϕ_1) and f2 (ϕ_2) from x axis and zenith angles (latitude) t1 (θ_1) and t2 (θ_2) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. dx*radius is then the difference between the two points in the x direction and d*radius is the total distance between the points.

```

double sphericalDistance(double f1, double t1,
    double f2, double t2, double radius) {
    double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
    double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
    double dz = cos(t2) - cos(t1);
    double d = sqrt(dx*dx + dy*dy + dz*dz);
    return radius*2*asin(d/2);
}
```

Various (10)

yCombinator.h

Description: Recursive lambdas without std::function.
Usage: auto fib = y_combinator([&](auto self, int n) -> ll { return n < 2 ? n : self(n-1) + self(n-2); });

```

template<class Fun> struct y_combinator_result {
    Fun fun_;
```

```
template<class T>
explicit y_combinator_result(T&& fun) : fun_(forward<T>(fun))
{}
template<class... Args> decltype(auto) operator()(Args&&...
args) const {
return fun_(ref(*this), forward<Args>(args)...);
}
};
template<class Fun> decltype(auto) y_combinator(Fun&& fun) {
return y_combinator_result<decay_t<Fun>>(forward<Fun>(fun));
}
```

10.1 Debugging tricks

- `signal(SIGSEGV, [](int) { _Exit(0); });`
converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). `_GLIBCXX_DEBUG` failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).
- `feenableexcept(29);` kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16).

10.2 Optimization tricks

`__builtin_ia32_ldmxcsr(40896);` disables denormals (which make floats 20x slower near their minimum value).

10.2.1 Bit hacks

- `x & -x` is the least bit in `x`.
- `for (int x = m; x; ) { --x &= m; ... }` loops over all subset masks of `m` (except `m` itself).
- `c = x&-x, r = x+c; ((r^x) >> 2)/c` | `r` is the next number after `x` with the same number of bits set.
- `rep(b,0,K) rep(i,0,(1 << K))`
`if (i & 1 << b) D[i] += D[i^(1 << b)];`
computes all sums of subsets.

10.2.2 Pragmas

- `#pragma GCC optimize ("Ofast")` will make GCC auto-vectorize loops and optimizes floating points better.
- `#pragma GCC target ("avx2")` can double performance of vectorized code, but causes crashes on old machines.
- `#pragma GCC optimize ("trapv")` kills the program on integer overflows (but is really slow).

FastMod.h

Description: Compute $a\%b$ about 5 times faster than usual, where b is constant but not known at compile time. Returns a value congruent to $a \pmod b$ in the range $[0, 2b)$.

```
typedef unsigned long long ull;
struct FastMod {
    ull b, m;
    FastMod(ull b) : b(b), m(-1ULL / b) {}
    ull reduce(ull a) { // a % b + (0 or b)
        return a - (ull)((__uint128_t(m) * a) >> 64) * b;
    }
};
```

```
};
};
```

FastInput.h

Description: Read an integer from stdin. Usage requires your program to pipe in input from file.

Usage: `./a.out < input.txt`

Time: About 5x as fast as `cin/scanf`.

7b3e70, 17 lines

```
inline char gc() { // like getchar()
    static char buf[1 << 16];
    static size_t bc, be;
    if (bc >= be) {
        buf[0] = 0, bc = 0;
        be = fread(buf, 1, sizeof(buf), stdin);
    }
    return buf[bc++]; // returns 0 on EOF
}

int readInt() {
    int a, c;
    while ((a = gc()) < 40);
    if (a == '-') return -readInt();
    while ((c = gc()) >= 48) a = a * 10 + c - 48;
    return a - 48;
}
```

MemoryPool.h

Description: Fast allocation of up to MAX objects of type T from a static buffer; memory is never freed. Beyond MAX it silently falls back to the heap. The pool holds `sizeof(T)*MAX` bytes, so declare it global/static. Needs GCC 9+ (memory_resource).

Usage: `MemoryPool<Node, 1<<20> pool; Node* x = pool.allocate();`

Time: $\mathcal{O}(1)$

2a1e2a, 13 lines

```
template<class T, int MAX>
struct MemoryPool {
    array<byte, sizeof(T) * MAX> buffer;
    pmr::monotonic_buffer_resource resource;
    pmr::polymorphic_allocator<T> allocator;
    MemoryPool() : resource(buffer.data(), buffer.size()),
        allocator(&resource) {}
    T* allocate(const T& val = T()) {
        T* p = allocator.allocate(1);
        allocator.construct(p, val);
        return p;
    }
};
```